



Module Code & Module Title

CS6P05NP & Final Year Project (Proposal)

Assessment Weightage & Type

FYP Interim Report

Title: "SkillXChange"

Student Name: Suraksha Sapkota

Email: Suraksha.sapkota.a23@icpe.edu.np

Student ID: 23049006

1st Supervisor: Sandip Dhakal

2nd Supervisor: Nischal Bhandari

Submission Date: 30th December 2025

Submission Due Date: 30th December 2025

I acknowledge that my proposal report must be submitted online through MY SECOND TEACHER on the appropriate module page before the assignment deadline to be accepted and assessed. I know that late submissions will be considered non-submissions, resulting in a zero mark.

Table of Contents

Table of Contents.....	2
Table Of Table	4
Table Of Figures	5
1. INTRODUCTION.....	1
1.1 Problem Scenario	1
1.2 The Project as a Solution.....	2
1.3 Aim and Objectives.....	2
1.4 Tools and Resources Required.....	3
2.1 Background Study.....	7
2.2 Similar Projects.....	8
2.2.1 LinkedIn Learning	8
2.2.2 Skillshare	9
2.2.3 Simbi.....	10
2.3 Journal Research.....	11
2.3.1 SWAP SKILL Platform	12
2.3.2 Skill_Fusion	13
2.3.3 Micro-payment exchange system.....	15
2.3.4 Microservices Architecture for Building a Crypto Freelance Exchange	16
2.4 Comparison	18
3. EXPECTED OUTCOMES AND DELIVERABLES	19
3.1 Expected Outcomes.....	19
3.2 Deliverables	19
4. PROJECT RISKS, THREATS, AND CONTINGENCY PLANS	20
5. METHODOLOGY.....	21
5.1 Model Overview	21
5.1.1 Waterfall Model.....	21
5.1.2 Prototype Model.....	22
5.1.3 Spiral Model.....	23
5.2 Selected Model	25
5.3 Work Breakdown Structure (Wbs).....	26
5.4 Milestones	27
5.5 Project Gantt Chart	28

6.	WORK DONE / PROOF OF WORK PERFORMED.....	29
6.1	Research and Requirement Analysis	29
6.1.1	LinkedIn Learning(https://www.linkedin.com/learning/)	29
6.1.2	SkillShare(https://www.skillshare.com/en/search/classes?query=freelance) 31	
6.1.3	Simbi(https://simbi.com/services/search)	33
6.1.4	PeerList(https://peerlist.io/scroll)	35
6.1.5	Mentorcruise	37
6.2	System Design.....	39
6.3	Wireframe	46
6.4	Implementation Work Completed	47
6.4.1	Code	48
6.4.2	Testing	51
6.5	System Requirement Specifications (SRS)	57
6.5.1	Introduction	57
6.5.2	Overall Description.....	57
6.5.3	Functional Requirements	58
6.5.4	Non-functional Requirements	59
6.5.5	Future Enhancements.....	60
7.	FURTHER WORK.....	61
7.1	Reflection on Changes from the Original Proposal.....	61
7.2	Remedial Plans Adopted.....	61
7.3	Further Work to be Done Using Incremental Development	62
7.4	Deployment and Documentation	63
7.5	Alignment with Proposed Gantt Chart	63
7.6	Expected Outcomes.....	63
8.	CONCLUSION	64
9.	REFERENCES.....	65

Table Of Table

Table 1: Compare my project to another project.....	18
Table 2: Project Risks, Threats, and Contingency Plans	20
Table 3: Milestones	27
Table 4: Test1 Registration with valid inputs.....	51
Table 5: Test2 Registration with empty fields	52
Table 6:Test3 Login with wrong credentials.....	54
Table 7: Test4 Login with correct credentials.....	54

Table Of Figures

Figure 1: Xampp.....	3
Figure 2: Laravel	3
figure 3: PHP.....	4
figure 4: VS Code.....	5
figure 5: MS word 365	5
figure 6: Figma	6
figure 7: Draw.io	6
figure 8: LinkedIn Learning's Home Page	8
figure 9: Skillshare.....	9
figure 10: Simbi	10
figure 11: Watterfall model.....	22
figure 12: Prototype model	23
figure 13: Spiral model	24
figure 14: Incremental Model.....	25
figure 15: Work Breakdown Structure	26
figure 16: Gantt chart.....	28
Figure 17: Linkedin learning	30
Figure 18: Wireframe of LinkedIn learning	31
Figure 19: Skill Share	32
Figure 20: wireframe of skillshare	33
Figure 21: simbi.....	34
Figure 22: wireframe of simbi	35
Figure 23: Peer List.....	36
Figure 24: wireframe of peerlist	37
Figure 25: Mentorcruise	38
Figure 26: wireframe of mentorcruise	39
Figure 27: Use Case Diagram	40
Figure 28: Class Diagram.....	41
Figure 29: ERD	42
Figure 30: Sequences Diagram.....	43
Figure 31: Flowchart.....	44
Figure 32: DFD 0.....	45
Figure 33: DFD1.....	45
Figure 34: Register page's wireframe	46
Figure 35: Login page's wireframe.....	47
Figure 36: login code from login.blade.php	48
Figure 37: Register code from register.blade.php	49
Figure 38: Migration file to create user	50
Figure 39: Route for login and register	50
Figure 40: Insert right value to successful registration	51
Figure 41: Testing with empty fields	52
Figure 42: Before Registration.....	53
Figure 43: After Registration.....	53

Figure 44: Login Page	53
Figure 45: Testing with incorrect values	54
Figure 46: Insert correct value for testing	55
Figure 47: Home page.....	55
Figure 48: user profile	56

1. INTRODUCTION

The title of the project, “SkillXchange: A Web-Based Skill Exchange Platform,” represents an innovative approach to learning and collaboration. The purpose of this project is to create a website where individuals can share their skills with one another for free. In today’s modern world, many individuals have valuable talents such as photography, music, programming, or designing, but they lack the funds to pay for professional courses. The SkillXchange platform is a fair, open, and rewarding place for teachers and trainees to meet and share their skills.

The system is designed as a responsive web application using the Laravel framework with PHP as the backend, HTML, CSS, and JavaScript for the frontend, and MySQL for the database. The Admin Dashboard, User Panel, and Skill Exchange Area are the platform's three primary areas, each with a distinct function to ensure seamless operation. While users can build profiles, display their abilities, issue barter requests, and grade other users after successful sessions, the administrator keeps an eye on everything, controls security, and makes sure the system is operating correctly. The SkillXchange platform is more than simply a website; it's a community-driven project that uses talent sharing to promote social engagement, self-learning, and personal growth. It incorporates premium membership features, AI-based matchmaking, and gamification to make the whole experience engaging, effective, and encouraging.

1.1 Problem Scenario

Learning new skills or obtaining expert services in today's fast-paced digital age frequently involves financial capabilities. Many people have abilities that they can contribute but cannot afford to take advantage of new learning possibilities, especially students, independent contractors, and jobless people. Current platforms lack a suitable framework for skill exchange and primarily concentrate on paid courses, freelance employment, or voluntary work. People lose out on the chance to work together and develop without worrying about money. Furthermore, it is challenging for users to locate compatible partners and uphold transaction transparency in Nepal due to the lack of a centralized system that offers a reliable, equitable barter mechanism for skill exchange.

1.2 The Project as a Solution

The Skill Barter System offers a practical solution by creating a digital platform where users can register, list their skills, and search for other users willing to exchange them. For example, someone with graphic design skills can trade their services for someone teaching programming. In order to preserve confidence, the platform guarantees safe authentication, mutual agreement verification, and feedback methods. An integrated messaging system allows users to interact with each other, explore postings, and suggest deals. By fostering economic inclusion, community cooperation, and skill development, this project enables people to advance both personally and professionally.

1.3 Aim and Objectives

Aim:

The aim of this project is to create an online platform that will allow people to quickly and equitably trade their abilities without the need for money. Through secure identification, AI-based matchmaking, gamification, and feedback mechanisms, the system will foster self-learning, communication, and community involvement.

Objectives:

- To create a secure login and registration system for users.
- To implement a skill management module where users can add “skills to teach” and “skills to learn.”
- To design an efficient matching algorithm for skill barter requests.
- To provide scheduling and chat features for session management.
- To develop a responsive interface using HTML, CSS, and JavaScript.
- To integrate a feedback and rating system for user reliability.
- To integrate a payment gateway for Premium Membership.
- To use the Incremental Software Development Model for phased delivery.

1.4 Tools and Resources Required

➤ Database: MySQL

XAMPP is an open-source local development environment created by Apache Friends. It includes Apache HTTP Server, MySQL Database, and interpreters for PHP and Perl, making it ideal for web development. MySQL will be used to manage all user data, skill records, and barter requests securely. XAMPP allows for easy setup and local testing before deployment (DSouza, 2024).



Figure 1: Xampp

➤ Framework: Laravel

The Model-View-Controller (MVC) design is used by the open-source PHP framework Laravel. It ensures organized and effective code by offering built-in features for database management, routing, and authentication. Laravel will be utilized in this project to handle data movement between the front end and back end, provide the fundamental features of the Skill Barter System, and uphold system security (tutorialspoint, 2024).



Figure 2: Laravel

➤ **Frontend: HTML, CSS, and JavaScript**

JavaScript adds logic and interactivity to improve user experience, CSS improves layout and design, and HTML establishes the webpages' structure. Together, they guarantee that the Skill Barter System is responsive, aesthetically pleasing, and simple to use on desktop and mobile platforms.



figure 3: HTML,CSS & JS

➤ **Backend: PHP**

PHP is a strong programming language that is used to manage backend operations including database communication, barter request processing, and user authentication. It ensures smooth operation between client and server, making the platform secure and dynamic (WillDom, 2023).



figure 3: PHP

➤ **Text Editor: Visual Studio Code**

Visual Studio Code, a free and open-source code editor, supports multiple programming languages. It facilitates the development of the Skill Barter System by providing features including real-time collaboration, debugging tools, and syntax highlighting (UCHENDU, 2021).



figure 4: VS Code

➤ **Documentation: Microsoft Word**

The Skill Barter System will use Microsoft Word to create reports, documentation, and proposals. It ensures that written documents are presented professionally by enabling structured formatting, editing, and integration of figures and tables.



figure 5: MS word 365

➤ **Design Tools: Figma and Draw.io**

System flowcharts, ER diagrams, and UML models will be produced using Draw.io (also called Diagrams.net). Diagramming project designs is made easier by its support for cloud storage and data privacy (Edirisinghe, 2023).

Figma will be used for UI/UX design. It is a collaborative online tool that allows for wireframing and interface design, helping create a modern and user-friendly look for the Skill Barter System (Purdila, 2023).



figure 6: Figma



figure 7: Draw.io

2. Literature Review / Background

2.1 Background Study

The background study of this project is the increased need to adopt flexible and affordable modes of learning in the digital era. Most of the existing online learning platforms are paid subscriptions, and this restricts access to students, freelancers and those who cannot afford them due to lack of money. This does not imply that many individuals have highly marketable skills that can be shared in case a suitable platform is available.

A skill bartering system is designed on the old barter system where money is not used in exchange of services. Such systems implemented in a digital form must have strong authentication, trust mechanisms, and transparent communication tools to eliminate fairness and transparency. Practical knowledge sharing, active participation, and collaboration are promoted by peer-to-peer learning that is based on skill exchange. Current web technologies allow creating safe and convenient platforms that facilitate such interactions. An exchange platform based on skills can encourage inclusive learning, community-accessed engagement, and self-improvement without any monetary limitations with the help of a web-based approach and proper system design.

2.2 Similar Projects

Several existing systems have inspired the development of the Skill Barter System. Platforms like LinkedIn Learning, Fiverr, and Skillshare focus on paid services, while platforms such as Simbi promote non-monetary exchange. However, these platforms often lack localized features suitable for Nepalese users, such as language options, community-based reputation systems, and easy barter matching. The Skill Barter System adopts the best features from these global models and tailors them for the local context.

2.2.1 LinkedIn Learning

Thousands of professionally taught online video courses are available on the well-known site LinkedIn Learning. Professional skill development and career promotion are its main priorities. Users can finish courses and use their LinkedIn profiles to highlight their accomplishments. It prevents users from interacting directly or exchanging skills. On the other hand, SkillXchange emphasizes free, peer-to-peer skill sharing, where users teach and learn from one another at no cost, encouraging collaboration and a sense of community.

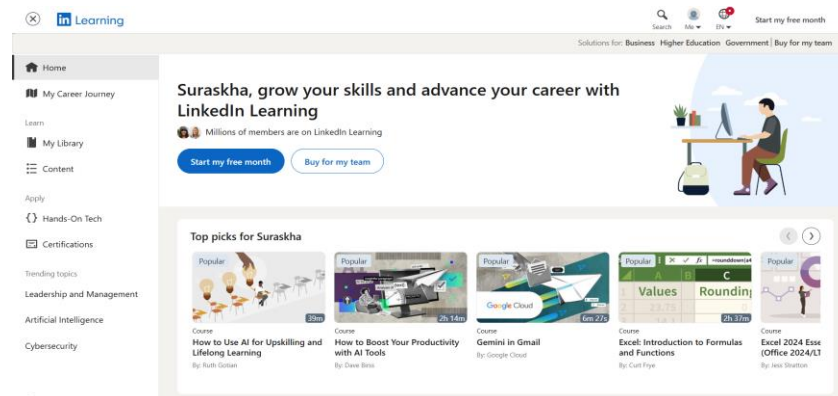


figure 8: LinkedIn Learning's Home Page

2.2.2 Skillshare

SkillShare is another global learning platform that allows experts to create and share video lessons in creative fields like design, photography, and writing. For those who cannot afford to pay for their education, access is limited because all classes require a subscription. SkillShare is focused on one-way learning from teachers to students and lacks teamwork or reciprocal communication. The SkillBarter platform, on the other hand, emphasizes two-way learning by enabling each user to act as both a teacher and a learner. It employs an AI-based matching engine to connect people with complementary abilities and a gamified system with badges and points to make learning fun and motivating.

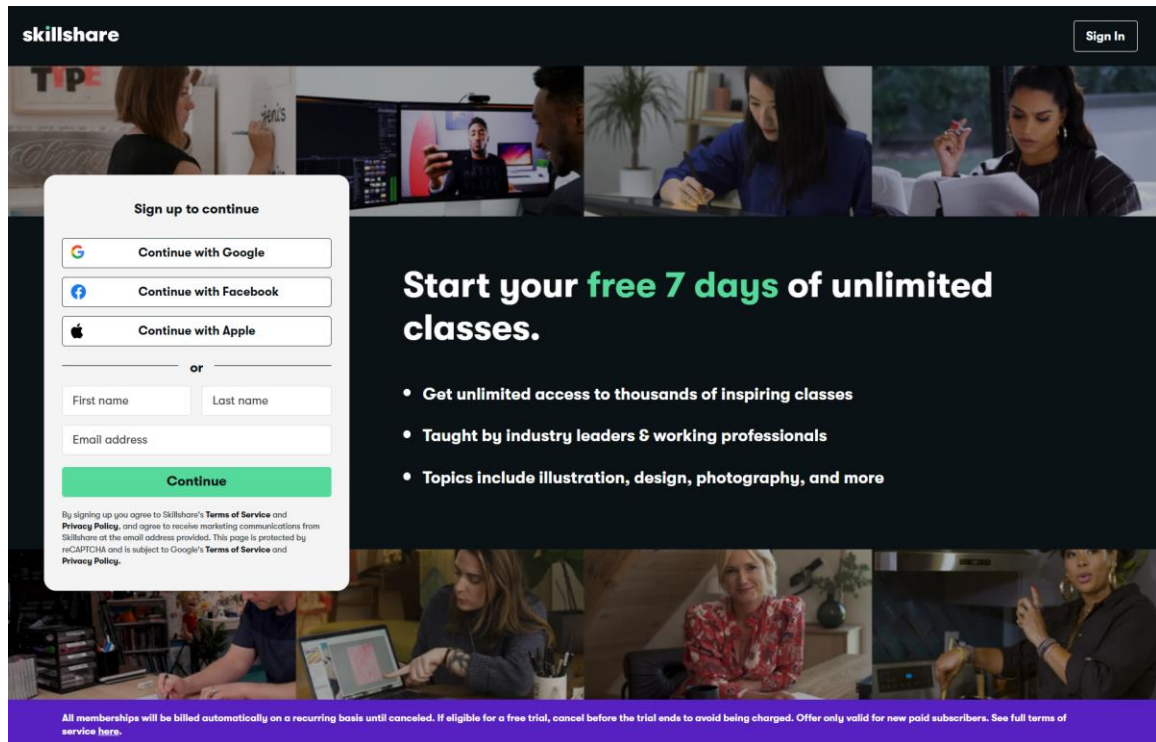


figure 9: Skillshare

2.2.3 Simbi

Simbi is one of the few platforms that allow users to trade services without using money. It works on a credit-based system where users earn credits by offering their skills and spending them to receive services from others. Although Simbi promotes non-monetary exchange, it lacks AI matchmaking, gamification features, and a localized design for users in Nepal. Simbi also does not provide built-in communication or scheduling tools. The SkillXchange system addresses these limitations by introducing an AI skill-matching algorithm, in-app chat and calendar scheduling, and Premium Membership features for users seeking enhanced experience.

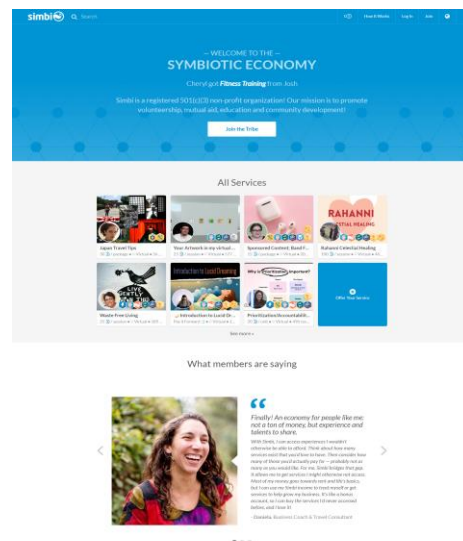


figure 10: Simbi

2.3 Journal Research

During the research phase, some academic journals and project-based studies regarding skill exchange, barter systems, and digital credit systems have been considered. The primary aim of this research was to learn more about the way non-monetary and credit-based exchange systems are structured and operated at the backend level.

The studies reviewed pointed to the application of credit, token, or point based systems to achieve equitable value exchange among users. They are systems that keep track of the credits that are earned and those that have been spent; they contain records of the transactions and have the administration level control in place to deter misuse or fraudulence. These kinds of mechanisms contribute to the establishment of trust, accountability and transparency in the platform.

Regarding business and engagement factors, the study revealed that the aspects of gamification, which include points, rewards, and loyalty systems, enhance user engagement and retention. Legally and security-wise, it is worth having records of the transactions and using authentication and access control to handle disputes and protect the data.

Another strength of the journals was the necessity of the presence of clear system actors and use cases, which are normally between users and administrators. Registration, skill management, matching, communication, and monitoring were the core functionalities that were shared by the studies. The study also assisted in determining the major system actors like users, skills, exchange, credits, and messages, which have contributed to the general design and database system of the system.

The journal study was a useful resource in the best practice and served to make sure that the system design is equivalent to the real-life, scalable and secure skill exchange platforms.

2.3.1 SWAP SKILL Platform

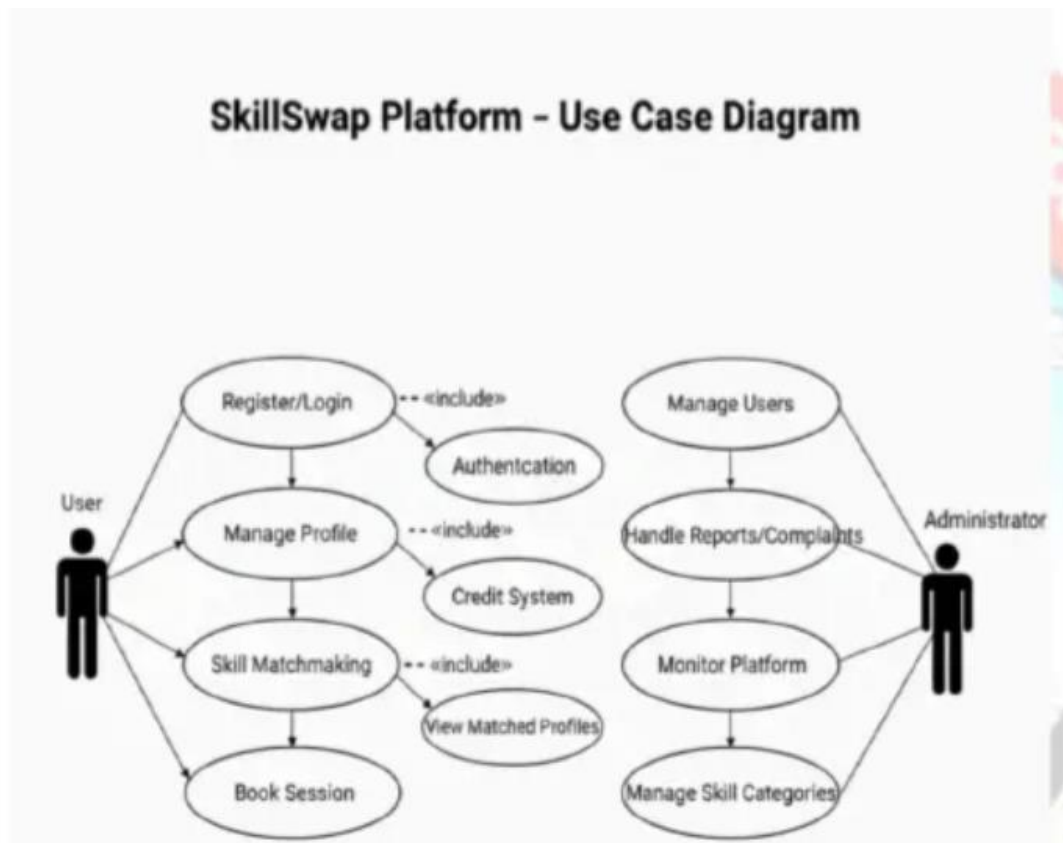
<https://www.scribd.com/document/943225058/Project-File-Swap-Skill>

Backend-Related Financial Components

- Token / credit-based barter engine.
- User account balances (credits earned & spent).
- Credit transfer logic triggered on session completion.
- Transaction log / ledger for every credit transfer.
- Admin-level monitoring for fraudulent or abnormal transfers.
- Feedback/rating storage affecting trust/credit privileges.

Business, Marketing & Legal Reasons for These Attributes

- Business: ensures fair value exchange; prevents freeloading.
- Marketing: tokens/gamification increase engagement and retention.
- Legal: transaction records support dispute resolution and audits.
- Compliance: need to secure financial-related user data and prevent abuse.



Actors include User and Admin, while the use cases include registration, login, skill management, matching, and chat.

2.3.2 Skill_Fusion

https://www.theijire.com/archiver/archives/skill_fusion.pdf

Backend-Related Financial Components

- Digital wallet (store skill-points or credits).
- Points issuance and deduction mechanisms.
- Reward/loyalty module for active contributors.
- Transaction ledger for points transfers.
- Authentication & role-based access for financial endpoints.

Business, Marketing & Legal Reasons for These Attributes

- Business: tracks value exchange and enables the platform economy.
- Marketing: loyalty/points encourage repeat participation.
- Legal: logs needed for disputes, fraud checks, and regulatory traceability.
- Security: authentication protects point balances and transfers.

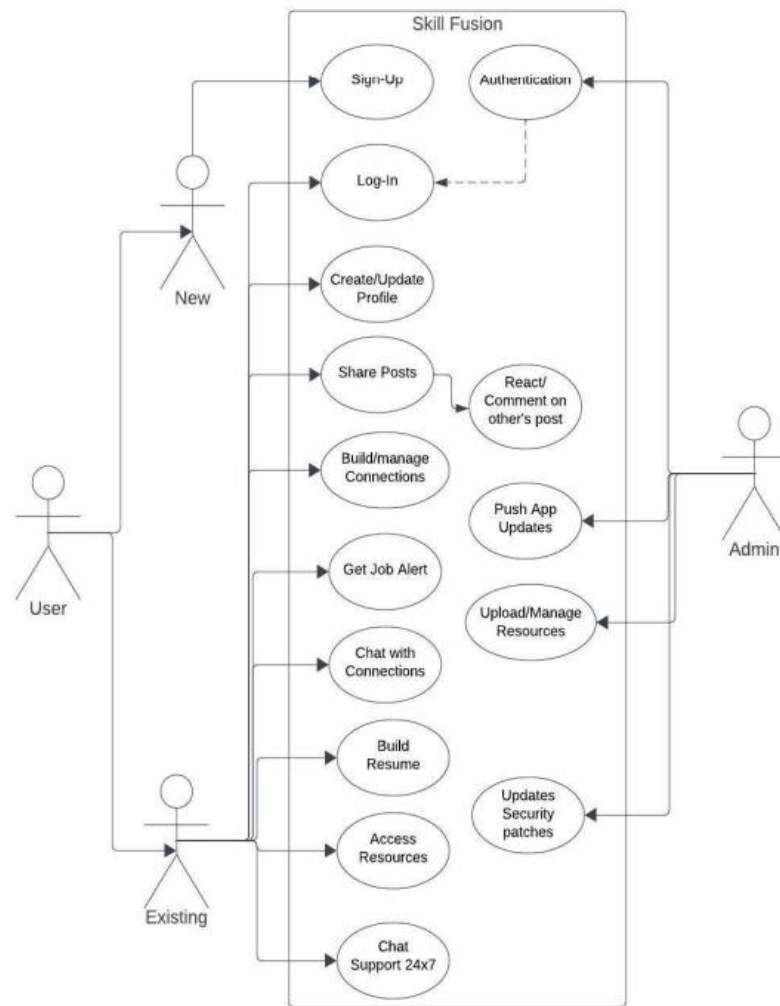


Figure 4: Use Case Diagram

Key Components of Use Case Diagrams:

- **Actors:** Represented as stick figures, actors are entities that interact with the system. They can be users, other systems, or external entities that participate in a use case.
- **Use Cases:** Shown as ovals, use cases outline the precise tasks or operations that the system carries out in reaction to an actor's interaction. Every use case illustrates a unique feature that benefits the actor.
- **System Boundary:** Shown as a rectangle, the system boundary encompasses all of the use cases that the system will address and defines its scope.

- **Relationships:** Associations between actors and use cases, such as generalization (inheritance), inclusion, and extension links, are depicted using lines and arrows, which aid in organizing and refining the interactions.

2.3.3 Micro-payment exchange system

<https://scholarworks.lib.csusb.edu/cgi/viewcontent.cgi?article=3855&context=etd-project&utm>

Backend-Related Financial Components

- E-wallet per user holding e-coins.
- Broker / Issuer module to mint and redeem e-coins.
- Transaction ledger / micro-payment records (sender, receiver, amount, timestamp).
- Secure transfer protocol to prevent double spending.
- Redemption & payout pipeline (convert coins to real currency).
- Audit & reconciliation services for coin issuance/redemption.

Business, Marketing & Legal Reasons for These Attributes

- Business: enables micro-transactions for small-value services/content.
- Marketing: small-cost payments broaden user base and monetization options.
- Legal: precise ledgers required for anti-fraud, AML/KYC where applicable.
- Compliance & privacy: protect users and meet financial regulations when redeeming coins.

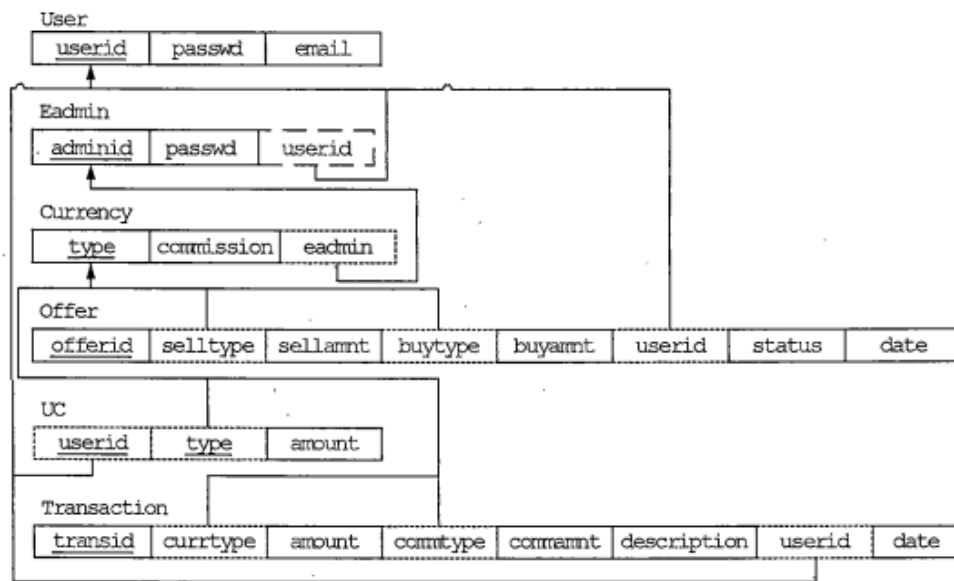


Figure 8. Relation Diagram For the Exchange Service Database

2.3.4 Microservices Architecture for Building a Crypto Freelance Exchange

<https://ceur-ws.org/Vol-4049/paper7.pdf?utm>

Business, Marketing & Legal Reasons for These Attributes

- Marketing Attributes: skill tags, provider reputation.
- Financial Attributes: token balances, conversion rates (token↔points).
- Legal Attributes: verification_status, identity docs reference.
- Analytics Attributes: matching_rate, service latency, success_rate_of_exchanges

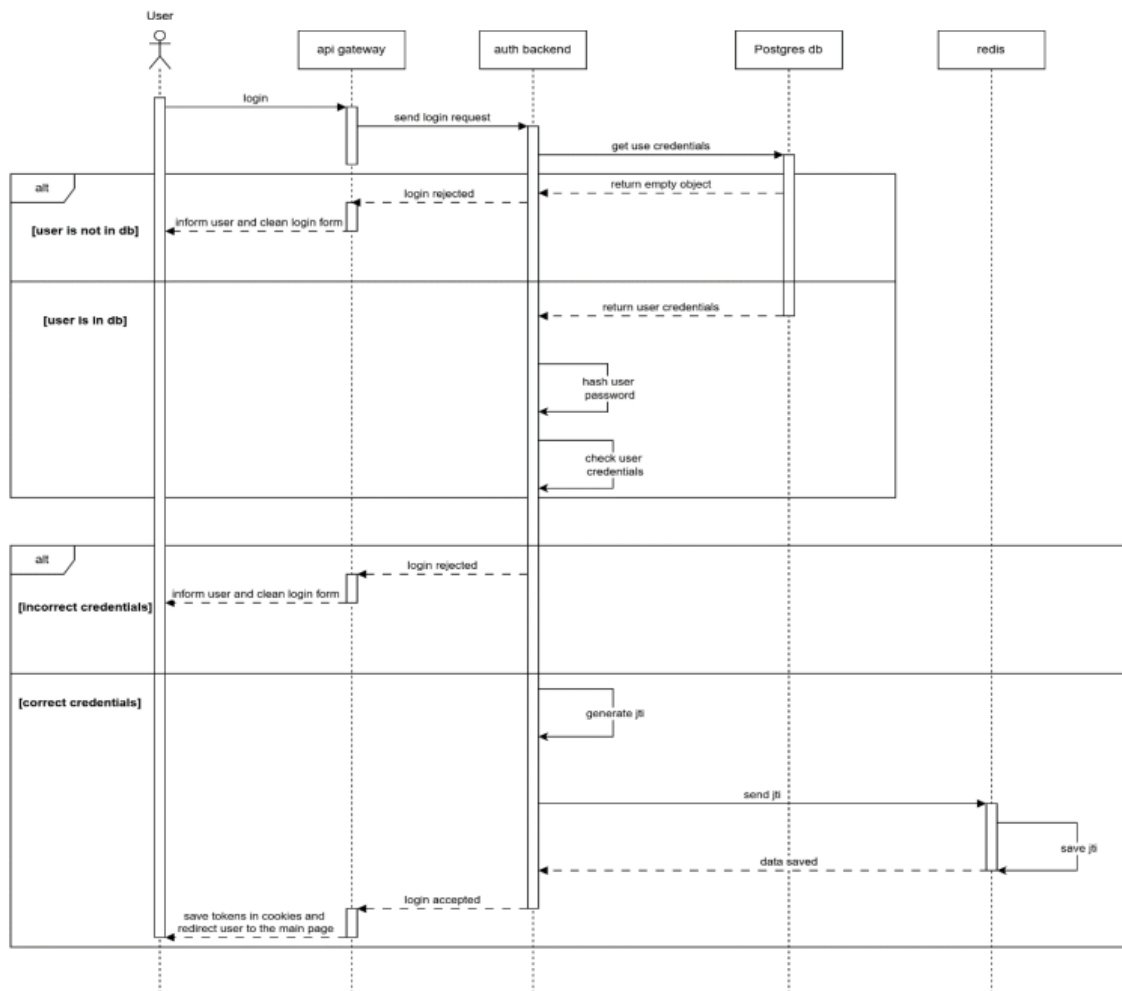


Figure 5: Sequence Diagram of User Authentication Flow

Entities / Attributes

- User Service: user_id, profile, credentials, verification_status
- Skill Service (catalog): skill_id, owner_id, name, category, description, level, metadata
- Match/Exchange Service: match_id, seeker_id, provider_id, matched_skill_ids, match_score, status
- Credit/Token Service: token_id, user_id, balance, earned, spent
- Notification / Messaging Service: message_id, from_id, to_id, content, timestamp

2.4 Comparison

The comparison below highlights the distinctions between similar systems and the proposed skill barter system.

Table 1: Compare my project to another project

Feature	LinkedIn Learning	SkillShare	Simbi	SkillXchange
Skill Exchange	-	-	✓	✓
Monetary Requirement	✓	✓	-	-
Rating & Feedback	✓	✓	✓	✓
Scheduling & Chat	-	-	-	✓
Gamification	-	-	-	✓
AI-Based Matching	-	-	-	✓
Premium Membership	✓	✓	-	✓
Peer-to-Peer learning	-	-	✓	✓

3. EXPECTED OUTCOMES AND DELIVERABLES

3.1 Expected Outcomes

- A fully functional web platform that supports skill exchange without monetary transactions.
- AI-based partner matching for fair and efficient skill barter.
- Improved user engagement through gamification and rewards.
- Premium users enjoy enhanced features like AI suggestions and resource uploads.
- A user-friendly interface promoting fair trade and interaction.
- Community development through collaboration and shared learning.

3.2 Deliverables

- Project proposal report
- System prototype and documentation
- Final project report and presentation
- Source code with user manual

4. PROJECT RISKS, THREATS, AND CONTINGENCY PLANS

Table 2: Project Risks, Threats, and Contingency Plans

Problems	Technical Issues	Security Breaches	Health Issues	Schedule Delays
Risk	Bugs or system crash	Unauthorized data access	Fatigue or illness	Missed milestones
Threat	Project delay	Loss of trust	Reduced productivity	Deadline extension
Plan	Regular code review and testing	Apply encryption and strong authentication	Maintain work-life balance	Adjust timeline and review progress weekly

5. METHODOLOGY

The development of the Skill Barter System will follow the Incremental Software Development Methodology. This model divides the project into smaller, manageable increments that are developed, tested, and refined in successive cycles. Each increment adds functionality to the system, ensuring continuous progress and early feedback. This approach suits the project well since it allows flexibility in requirements and iterative improvements based on user testing.

5.1 Model Overview

5.1.1 Waterfall Model

The Waterfall Model, introduced by Winston W. Royce in 1970, follows a simple step-by-step development process. The phases include Requirement Analysis, System Design, Implementation, Testing, Deployment, and Maintenance. Each phase is completed before moving to the next one. This model is best for projects where requirements are clearly defined from the beginning and do not change frequently.

Advantages:

- Fixed timeline with well-defined phases.
- Easy to manage since each phase has clear deliverables.
- Good for small projects with predictable requirements.

Disadvantages:

- It is difficult to handle unexpected changes during development.
- Testing happens late, which may increase cost if major errors appear.
- Less flexibility and limited user feedback during early stages.

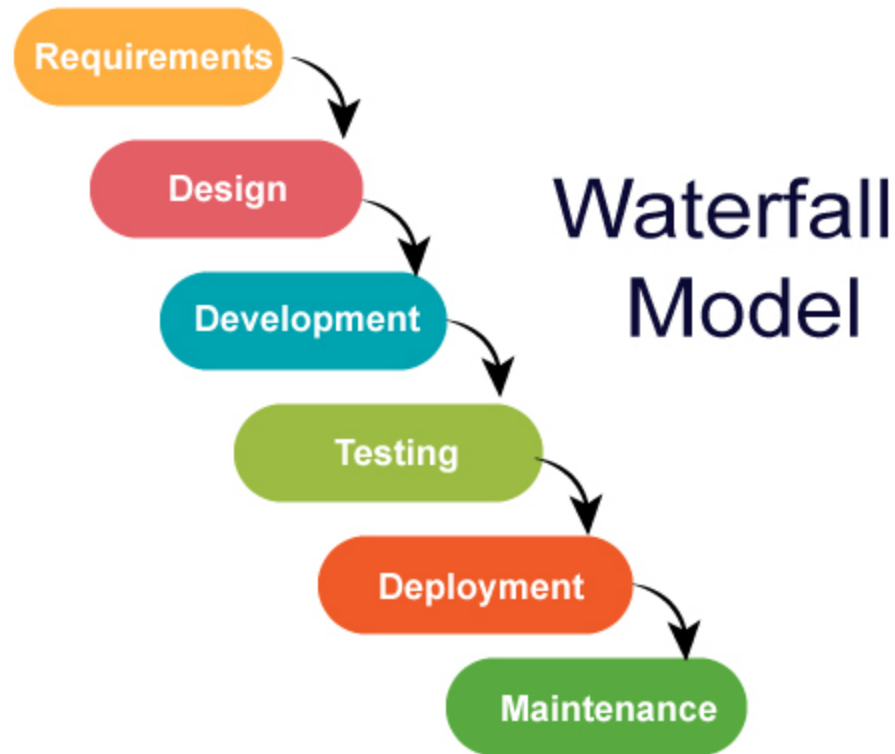


figure 11: Watterfall model

5.1.2 Prototype Model

The Prototype Model focuses on building a quick prototype (or demo version) of the system before the final one is developed. This helps users understand what the system will look like and gives feedback early. The cycle includes Requirement Gathering, Quick Design, Prototype Development, Testing, and Refinement until the system meets user expectations.

Advantage:

- Encourages user involvement and feedback.
- Helps visualize the system early in development.
- Increase customer satisfaction because users see progress.

Disadvantages:

- Increase time and cost due to repeated refinements.
- Create confusion between the prototype and the final system.
- Lead to incomplete documentation.

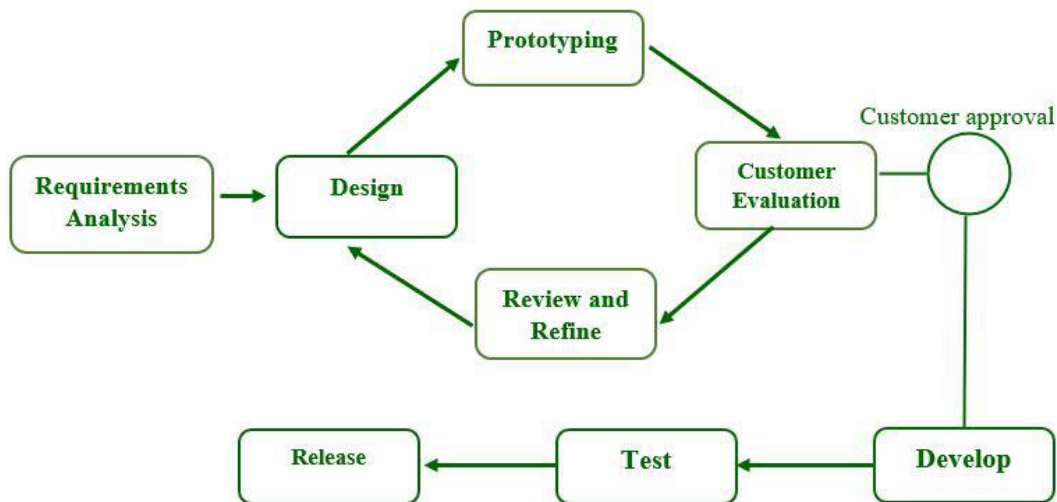


figure 12: Prototype model

5.1.3 Spiral Model

The Spiral Model combines features of both the Waterfall and Prototype models. It is used for large or complex projects that need continuous improvement and risk management. The process repeats in cycles called *spirals*, where each loop includes Planning, Risk Analysis, Engineering, Testing, and Evaluation.

Advantages:

- Focus on risk management and problem-solving.
- Suitable for complex and long-term projects.
- Allows continuous refinement and customer input.

Disadvantages:

- Costly and time-consuming for small projects.
- Hard to manage due to complexity.
- Need experienced developers for risk assessment.

SPIRAL MODEL IN SOFTWARE DEVELOPMENT

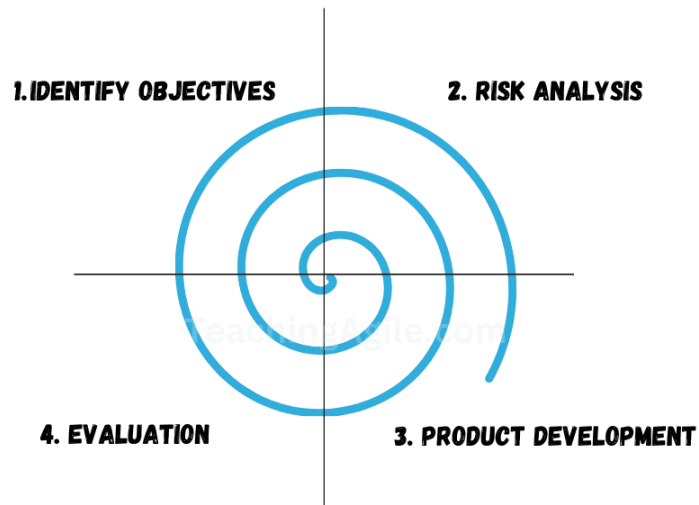


figure 13: Spiral model

5.2 Selected Model

The incremental model in software engineering divides a project into smaller modules, or increments, developed and integrated gradually. This iterative approach enables frequent testing, quick feedback, and early detection (Sachan, 2024).

The Incremental Model is a great fit for this project because it breaks the work into smaller steps, making development and integration easier. Each part is tested and improved before moving on, which helps create a more stable and unified system. This method also allows for regular feedback and makes it easier to adjust to changes, ensuring the final product meets user needs and is strong and reliable.

Advantages of the Incremental Model:

- Phased Development: Divides work into smaller and clearer stages.
- Early Integration: Delivers working parts early for testing.
- Reduced Risk: Errors can be found and fixed early.
- User Feedback: Feedback collected after each increment improves quality.
- Steady Progress: Development continues smoothly without delays.
- Adaptability: Easier to adjust features based on user needs or new ideas.

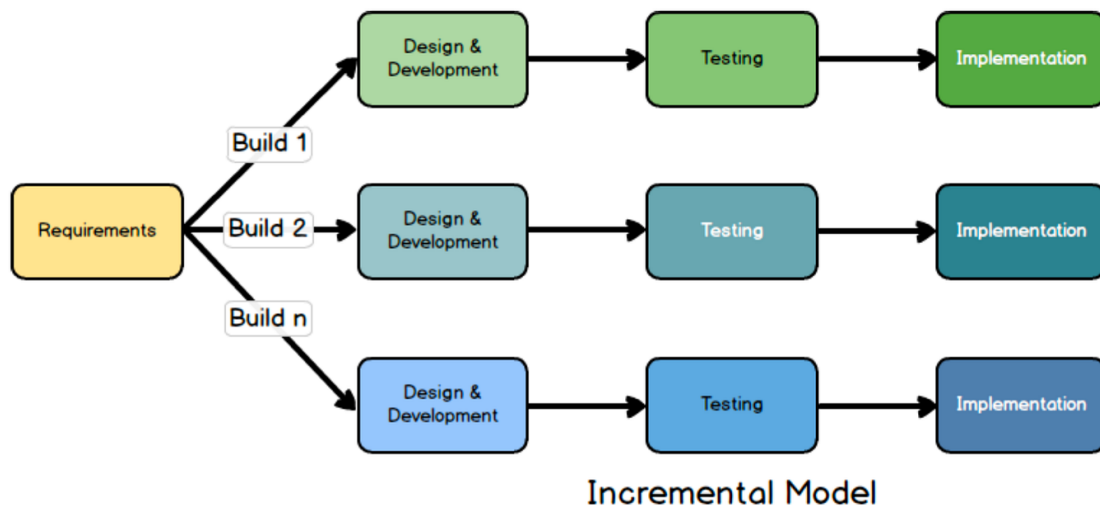


figure 14: Incremental Model

5.3 Work Breakdown Structure (Wbs)

The following breakdown represents the major phase of development using the Incremental Model:

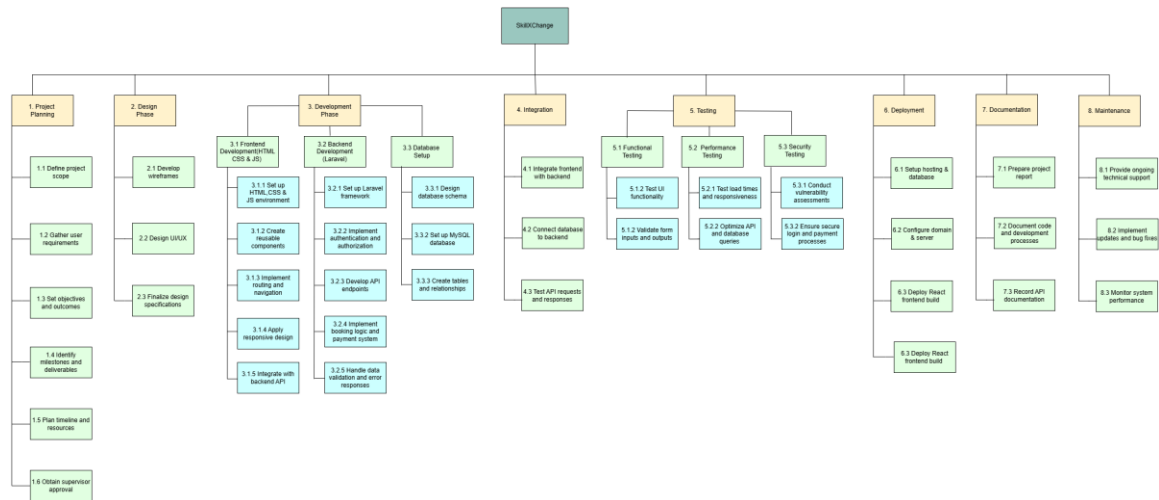


figure 15: Work Breakdown Structure

5.4 Milestones

Table 3: Milestones

Milestones	Activities	Dates
1	Finalize Topic	25 July 2025 to 08 Aug 2025
2	Topic Selection	09 Aug 2025 to 14 Aug 2025
3	Gather Requirements	15 Aug 2025 to 05 Sep 2025
4	Design Approval	06 Sep 2025 to 10 Oct 2025
5	Proposal Report Submission	11 Oct 2025 to 14 Nov 2025
6	Frontend Development	15 Nov 2025 to 10 Dec 2026
7	Backend Development	11 Dec 2025 to 10 Jan 2026
8	Database Setup	11 Jan 2026 to 10 Feb 2026
9	Integration Testing	12 Feb 2026 to 25 Feb 2026
10	Release Alpha Testing	26 Feb 2026 to 03 Mar 2026
11	Beta Testing	04 Mar 2026 to 15 Mar 2026
12	User Feedback Collection	16 Mar 2026 to 25 Mar 2026
13	Refine Features	26 Mar 2026 to 30 Mar 2026
14	Final Testing	01 Apr 2026 to 08 Apr 2026
15	Deployment	09 Apr 2026 to 15 Apr 2026
16	Submit Documentation	16 Apr 2026 to 26 Apr 2026
17	Review Project	27 Apr 2026 to 02 May 2026
18	Submit Final Report	03 May 2026
19	Prepare Presentation	04 May 2026
20	Implement Feedback	05 May 2026
21	Close Project	06 May 2026
22	Final Evaluation	07 May 2026

5.5 Project Gantt Chart

The Gantt Chart below represents the timeline of the Skill Barter System project across the current academic term. Each bar corresponds to the time allocated for major project milestones.

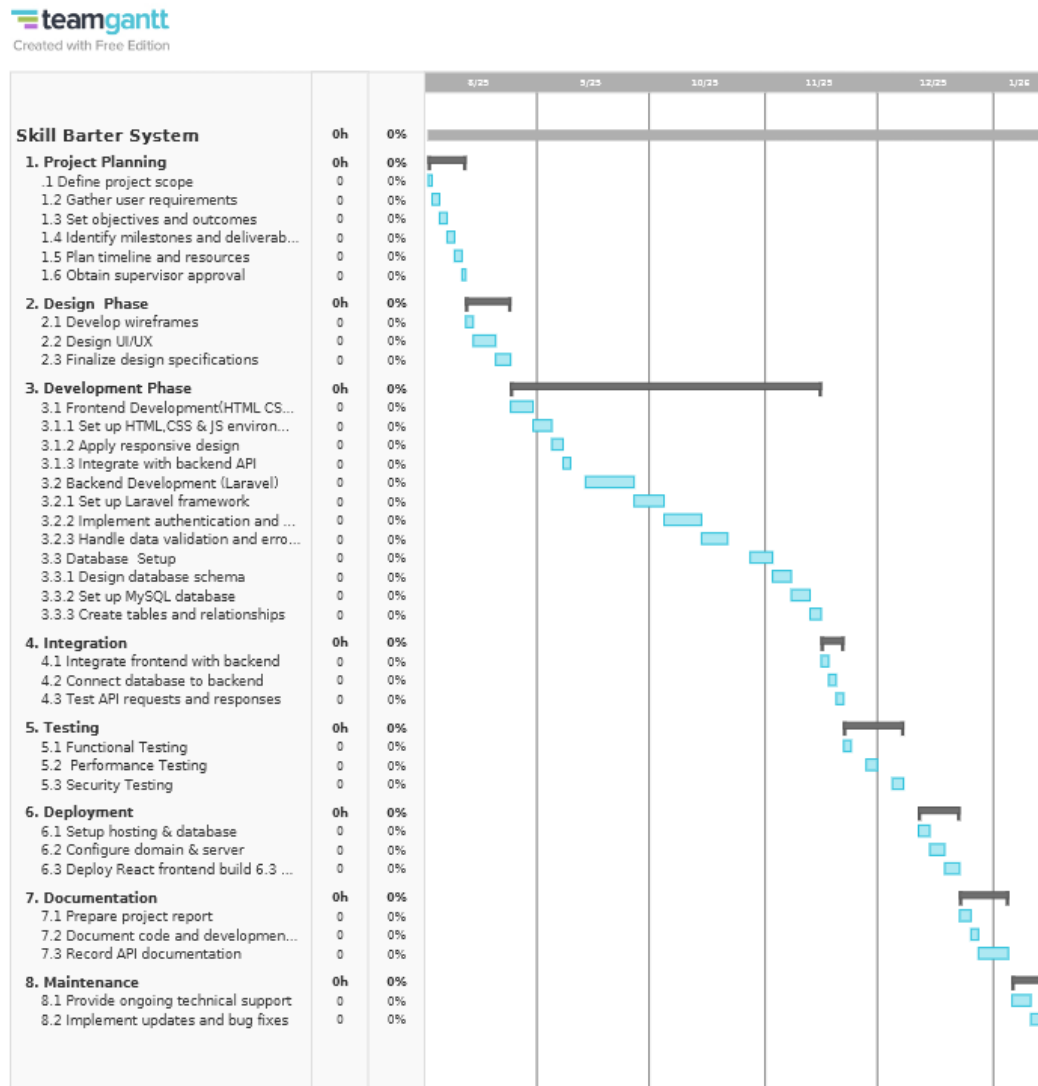


figure 16: Gantt chart

6. WORK DONE / PROOF OF WORK PERFORMED

The following work has been completed to date, aligned with the Incremental Model phases:

6.1 Research and Requirement Analysis

In the research and requirement analysis, several analogous skill-based and learning web sites were studied in detail to have an idea of typical system designs, user experience trends, and functional expectations. The study was primarily concerned with the way these platforms are structured, how they display filtering options, and help the user navigate with ease as he or she goes through skills or learning materials. Focus was directed at the analysis of front-end layout patterns, specifically, the application of organized sections, where filtering and category settings must be located to the one side of the interface, and the actual results of the content are located to the other side. This design strategy was found to be suitable for enhancing usability, refinement of search, and general user experience. A high-level review of the popular modern web technologies was also undertaken in the research in order to have a glimpse of how performance, responsiveness and scaling is established in real systems.

According to the results of this study, system requirements were found and developed. With the help of it, functional requirements, such as skill browsing, filtering, and presentation of content in a structured way, were obtained, as well as non-functional ones, including responsiveness, usability, and security. The UI designs and wireframes developed on the project were also based on these patterns and modified to apply to a skill barter system environment. In this research, practicality, user oriented and industry standards were put in consideration to make the system requirements and design decisions practical.

6.1.1 LinkedIn Learning(<https://www.linkedin.com/learning/>)

Front-end:

- Ember.js (JavaScript framework)
- Core-JS (JavaScript polyfill library)
- Custom UI components (LinkedIn's internal design system)

Back-end:

- Not public

Security:

- HSTS (HTTP Strict Transport Security)
- Cloudflare Bot Management (protection against automation & threats)

CDN:

- Cloudflare

Overview:

LinkedIn Learning uses a highly optimized enterprise-level tech stack designed for speed, security, and heavy user traffic. The presence of Ember.js indicates that the platform uses a single-page application structure, enabling smooth page transitions and fast navigation.

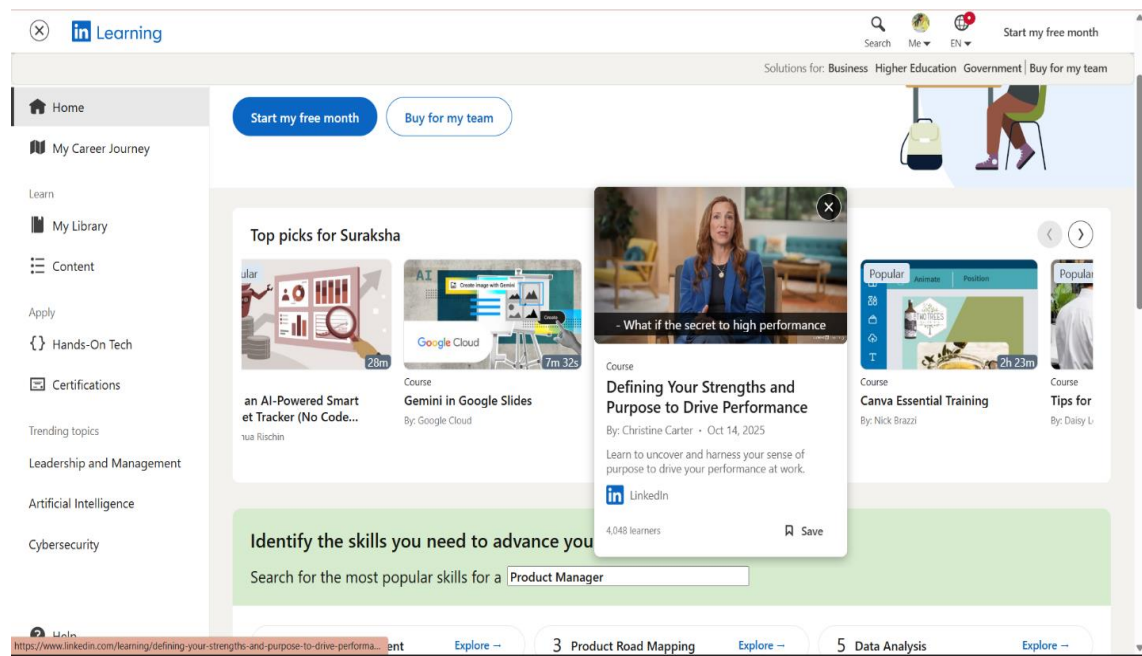


Figure 17: LinkedIn learning

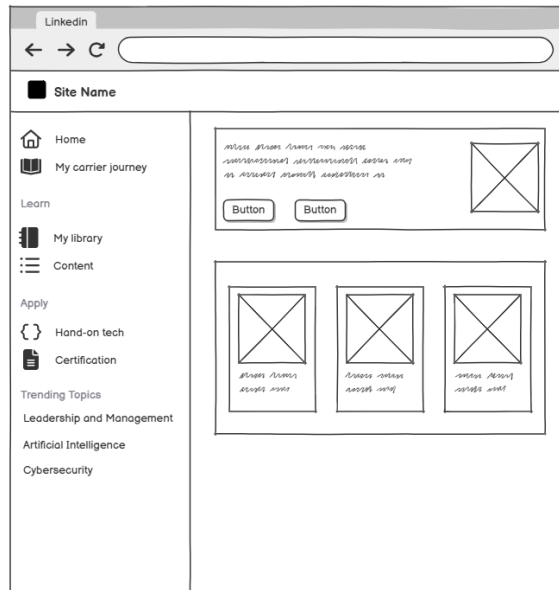


Figure 18: Wireframe of LinkedIn learning

Features I Liked

The platform implements a well-structured card-based listing system. Each course card initially displays a thumbnail and title, while additional metadata becomes visible on hover through a scale transformation. This interaction pattern minimizes visual clutter while ensuring that key information, such as course summary, save action, and learner count, is accessible without leaving the list view.

6.1.2 SkillShare(<https://www.skillshare.com/en/search/classes?query=freelance>)

Front-end:

- React
- Next.js 12.3.4 (SSR + SSG framework)
- Svelte (used in some modules)
- JSS
- Emotion
- styled components
- Goober (CSS-in-JS library)
- Material UI (UI components)

Back-end:

- PHP (detected through Yii framework)
- Next.js server-side rendering
- AWS (hosting + compute services)

JavaScript Libraries:

- Lodash 4.17.21
- core-js 3.0.1
- jQuery (legacy usage)

Security:

- Cloudflare Bot Management
- HSTS (HTTPS enforcement)

CDN + Server:

- Cloudflare (caching, CDN, security)
- Next.js server environment
-

Overview:

Skillshare uses a hybrid, modern architecture combining React + Next.js on the front-end with a PHP-based Yii framework on the back-end — showing that the platform is partially migrated to modern infrastructure.

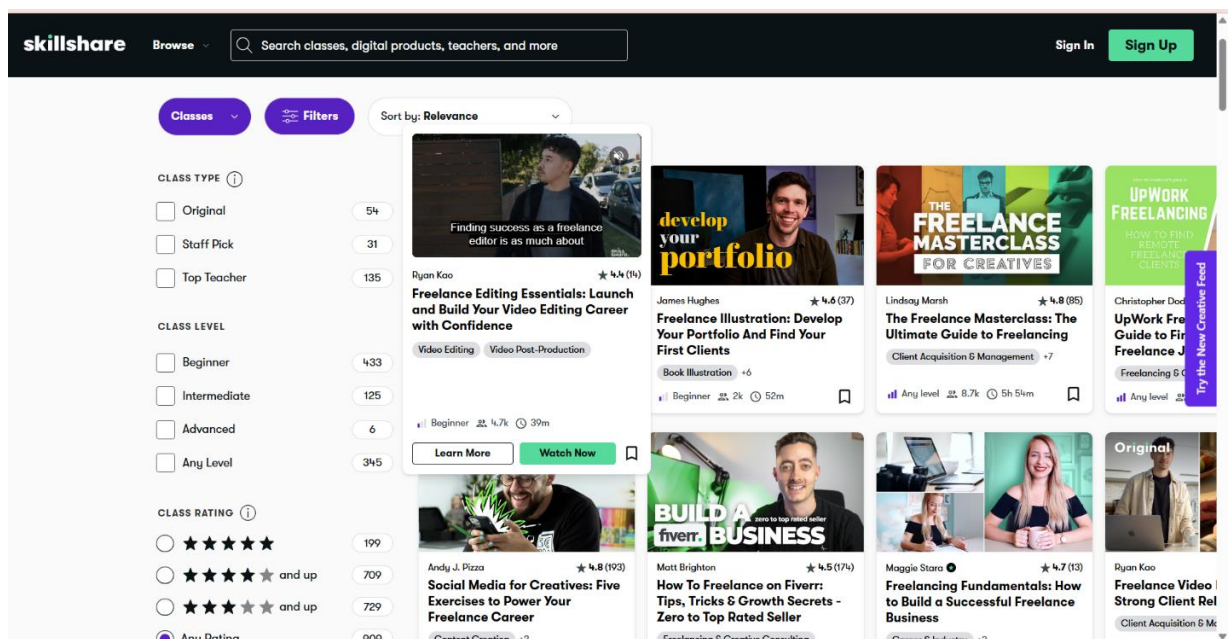


Figure 19: Skill Share

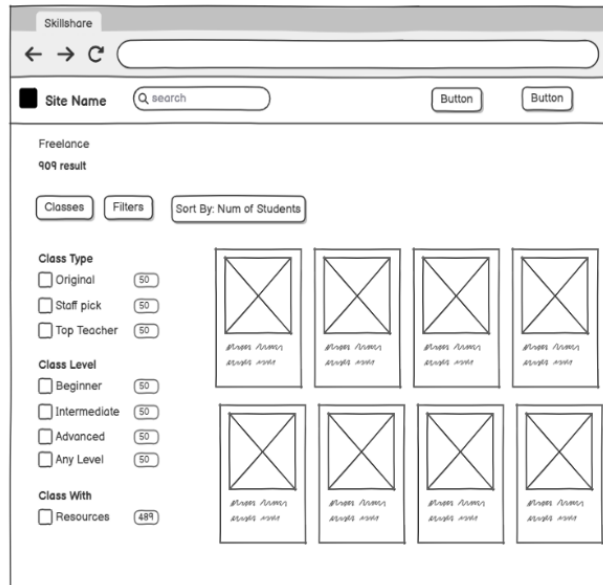


Figure 20: wireframe of skillshare

Features I liked:

Skillshare provides an efficiently designed browsing layout with filters positioned on the left panel (class type, level, rating) and corresponding course results displayed on the right. This two-column structure enhances discoverability, supports quick refinement of results, and maintains a consistent, visually balanced interface.

6.1.3 Simbi(<https://simbi.com/services/search>)

Front-end:

- Vue.js (primary frontend framework)
- jQuery 4.0.1 (legacy use)
- Lodash 4.17.21
- Hammer.js 2.0.7 (touch gestures)
- core-js 3.32.2 (JS polyfills)
- Bootstrap 3.4.1 (UI layout + components)

Back-end:

- Ruby (primary language)
- Ruby on Rails (main backend framework)
- AWS (hosting infrastructure / PaaS)

Security:

- HSTS (Strict HTTPS Enforcement)

CDN:

- Cloudflare
- cdnjs (JS library delivery)

Overview:

Simbi is built on a classic Ruby on Rails backend paired with a modern Vue.js frontend, providing a balanced architecture that combines reliable backend logic with a responsive, component-based UI.

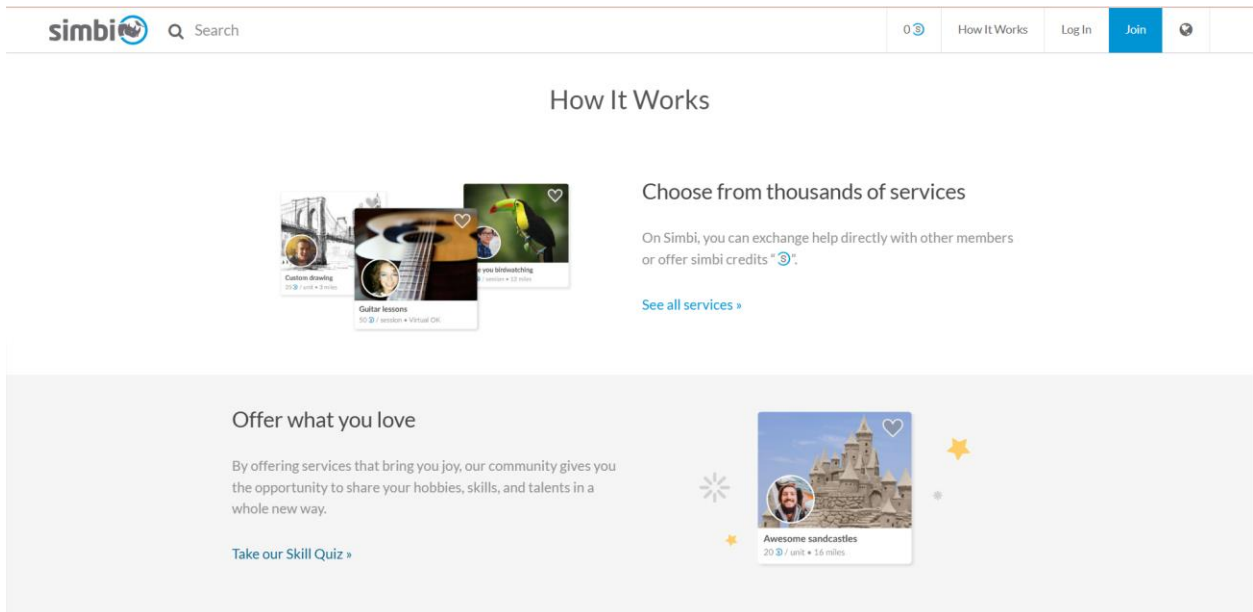


Figure 21: simbi

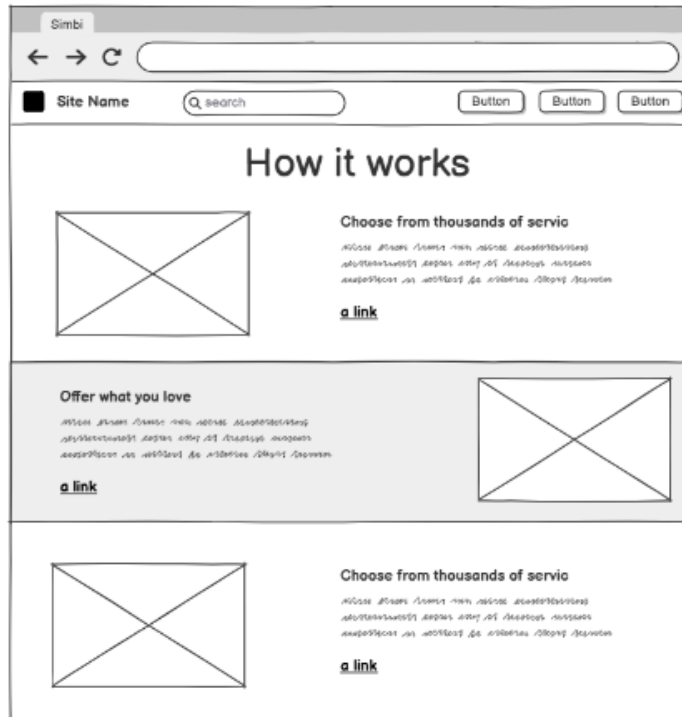


Figure 22: wireframe of simbi

Featured I liked:

Simbi clearly defines its workflow through a dedicated “How It Works” section supported by contextual links for user guidance. The “Offer What You Love” module is particularly effective as it includes an interactive quiz that helps users identify relevant skills, addressing uncertainty and improving onboarding quality.

6.1.4 PeerList(<https://peerlist.io/scroll>)**Front-end:**

- React (primary JavaScript framework for interactive UI)
- Next.js 15.2.3 (modern React-based framework for SSR, SSG, and routing)
- Tailwind CSS (utility-first CSS framework for fast styling)
- Framer Motion (advanced motion & animation library for UI transitions)
- core-js 3.25.5 (JavaScript polyfills)
- LottieFiles 5.10.2 (animated icons and illustrations)

Back-end:

- Next.js 15.2.3 (also used as backend API routes)
- PHP

Database:

- My SQL

Security:

- HSTS (forces secure HTTPS)
- NextAuth.js (secure authentication framework)
- Google Sign-in (OAuth-based authentication)

Overview:

This platform uses a modern Next.js + React stack combined with Tailwind CSS and Framer Motion for fast, animated, and highly responsive UI.

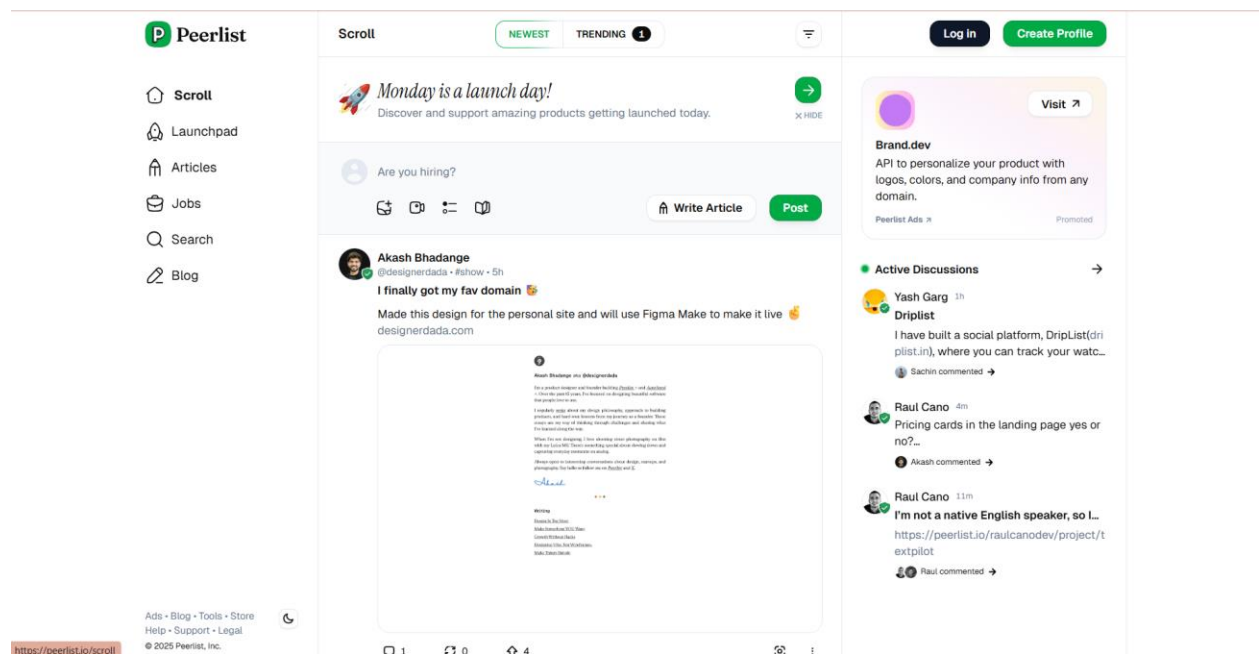


Figure 23: Peer List

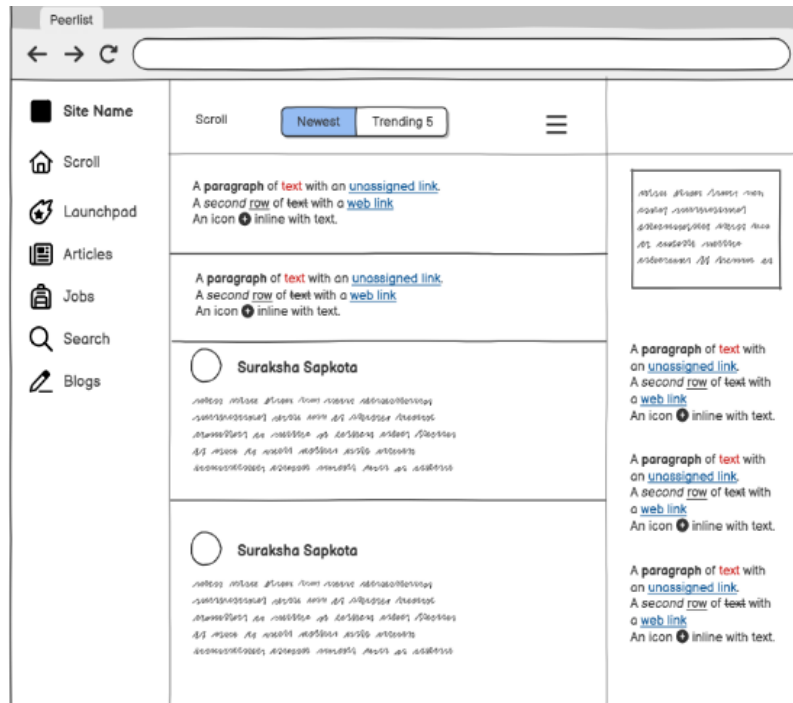


Figure 24: wireframe of peerlist

Featured I liked:

Peerlist has a unique feature where users can share short, useful posts like tips, ideas, and lessons. It also lets people read articles written by other professionals. This makes the platform great for learning and sharing knowledge in a simple way.

6.1.5 Mentorcruise

Front-end:

- React (primary modern JavaScript framework)
- Alpine.js 3.11.1 (lightweight framework used for interactive UI behaviors)
- core-js 3.32.2 (JavaScript polyfills for compatibility)
- jQuery 3.6.0 (legacy script usage for DOM manipulation)

Back-end:

- Django (Python framework) powering dynamic backend logic
- Python as main backend development language
- PHP (used for WordPress-driven sections of the platform)

Security:

- Cloudflare Turnstile (modern captcha alternative for bot protection)
- HSTS (Strict HTTPS for secure communication)

- Cookie Control + Cookiebot 1 (cookie consent + GDPR compliance)
- Cloudflare (DDoS protection, firewall, SSL)

Overview:

This platform uses a hybrid technology architecture, combining:

- Django (Python) for backend application logic
- WordPress (PHP) for CMS content
- React + Alpine.js + HeroUI for responsive UI components
- MySQL for structured data storage

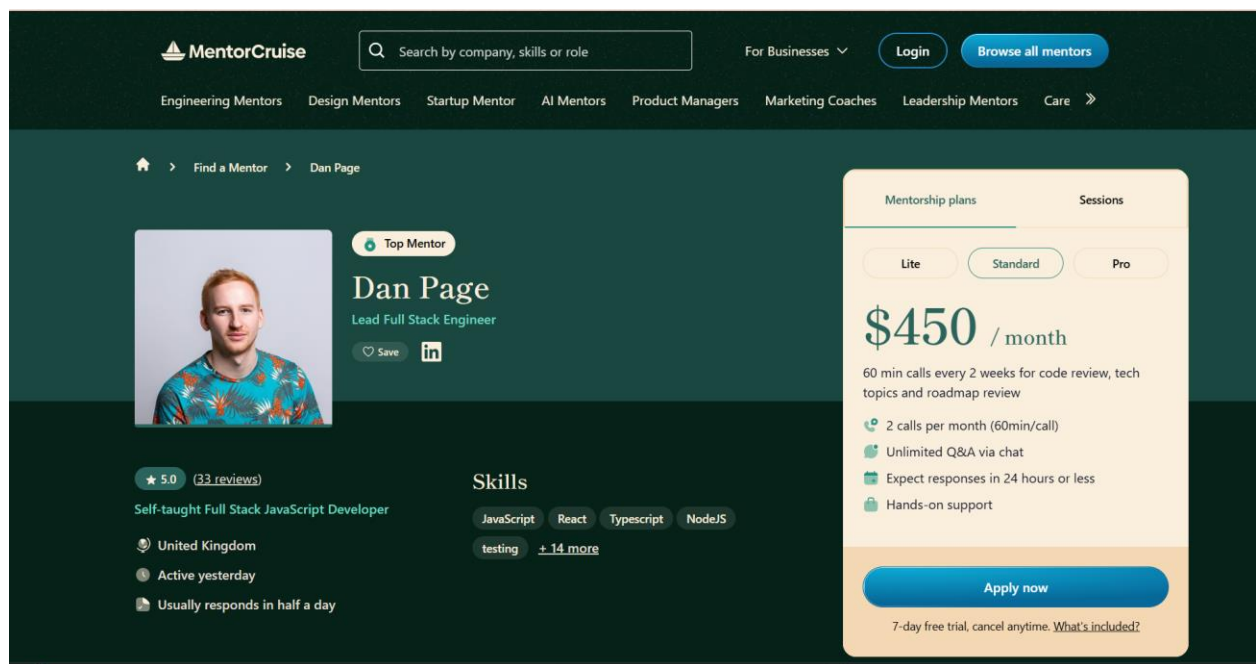


Figure 25: Mentorcruise

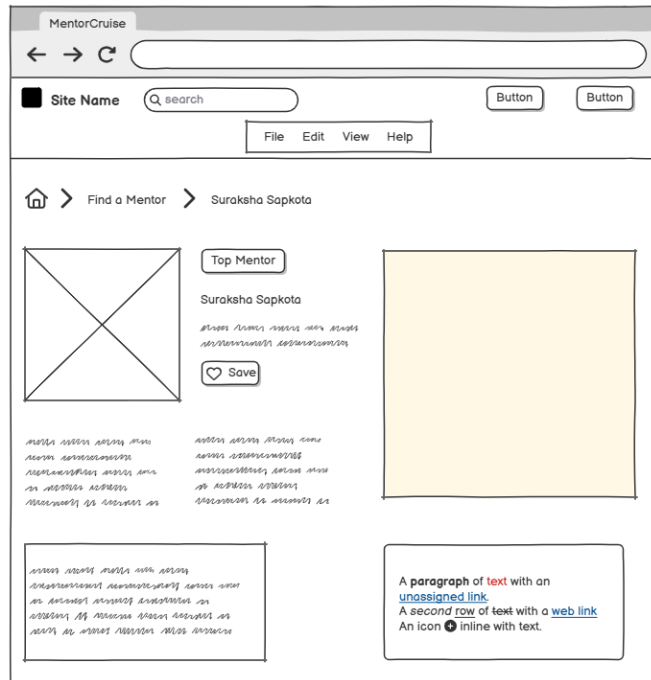


Figure 26: wireframe of mentorcruise

Featured I liked:

The mentor detail page follows a highly optimized information layout, integrating mentor profile images, teachable skills, background description, and user reviews in a single view. This clear hierarchy of information supports efficient decision-making and improves overall discoverability of mentor expertise.

6.2 System Design

There are different diagrams on system design and modeling, which were developed to gain an insight and design of the system. The system operation, user interaction with the system, data flow and database structure are visualized using these charts. UML diagrams composed of Use Case, Class, Sequence and Collaboration were used to model system functionalities, structure and interaction between the different components of a system. To expose data flow within the system, Data Flow Diagrams (DFD) were created. To design the database structure, an Entity Relationship Diagram (ERD) was drawn, and a system flowchart was provided to illustrate the process flow and the logic behind decision making in general. These diagrams helped to make the system design understandable and to implement it on a good basis.

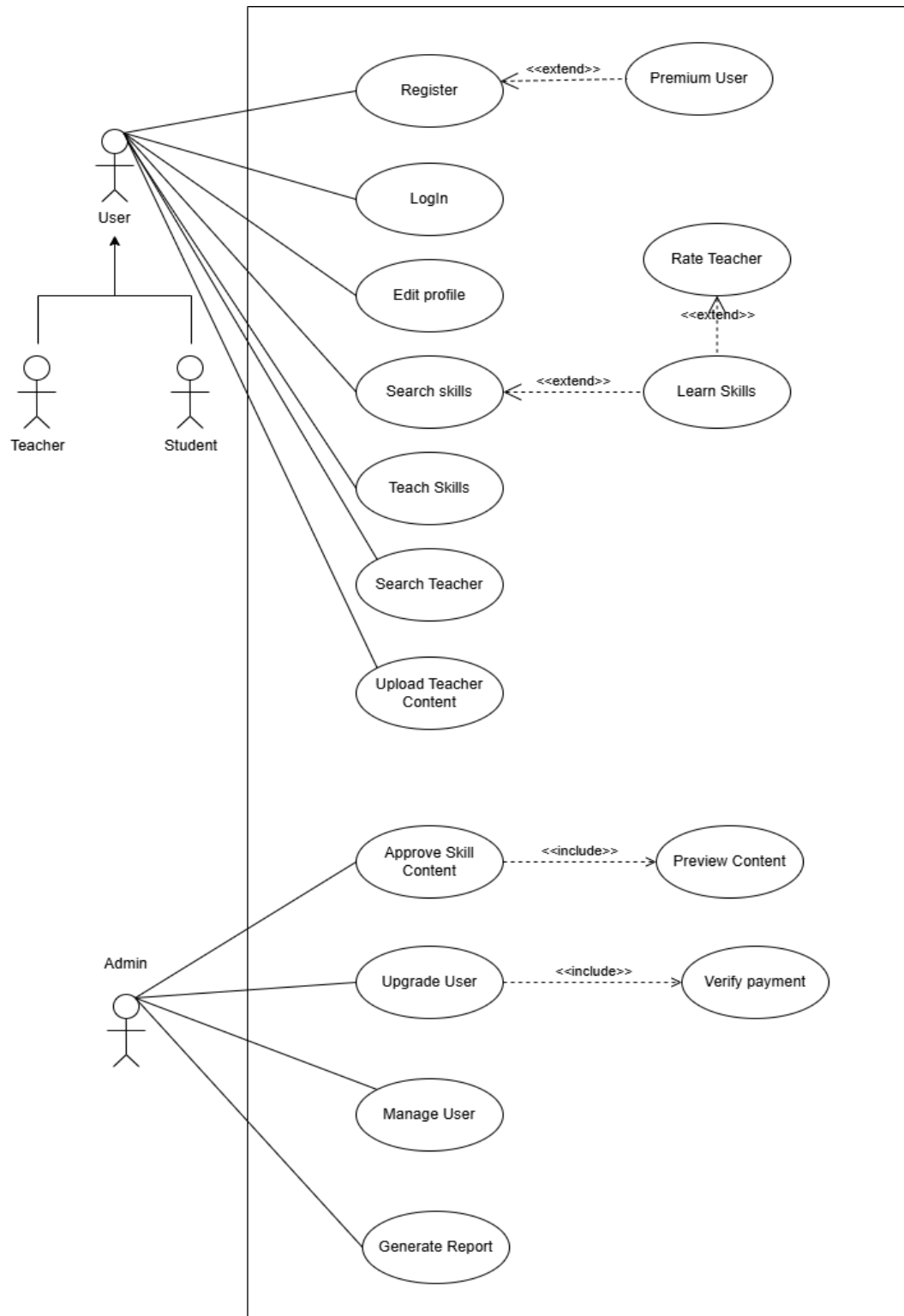
Use Case Diagram:

Figure 27: Use Case Diagram

The use case diagram describes the functional requirement of the Skill Barter System in terms of how various actors interface with the system. User is the primary actor that is further specialized into Student and Teacher who have similar features like registration, login, profile management, and search of skills. There are other applications such as teaching skills and uploading learning content by the teachers and the option of rating teachers after learning is available to the user. The Premium User is presented as an optional add-on to registration, where one can upgrade to additional features. The admin handles system level activities including skill content approval, user upgrades, payment verifications, user administration and report generation with a well-defined separation of user and administrative duties based on include and extend relationships.

Class Diagram:

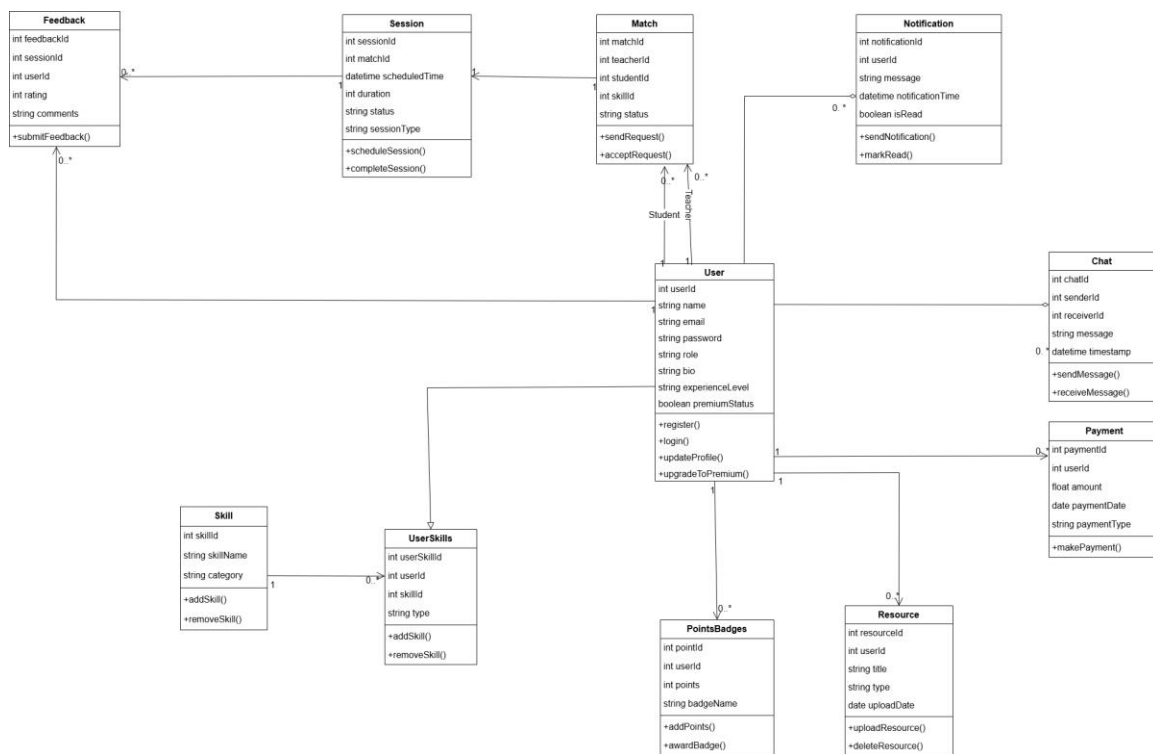


Figure 28: Class Diagram

The class diagram is an object-oriented design of the system that displays the arrangement of functionality into classes. The User class is the central one, but it handles authentication, profile, and premium upgrades. Skills are also handled using the Skill and UserSkills classes, where the users are connected to pedagogical roles or to the role of

learning. The classes are Match and Session that handle requests and scheduled learning activities and the Chat that facilitates messaging between the users. Resource process uploaded learning content, Payment is used to handle transactions, Notification sends system notification and Feedback receives session ratings. In general, the diagram shows that there is a distinct division of responsibilities, which contributes to scalability and maintainability.

Entity Relation Diagram:

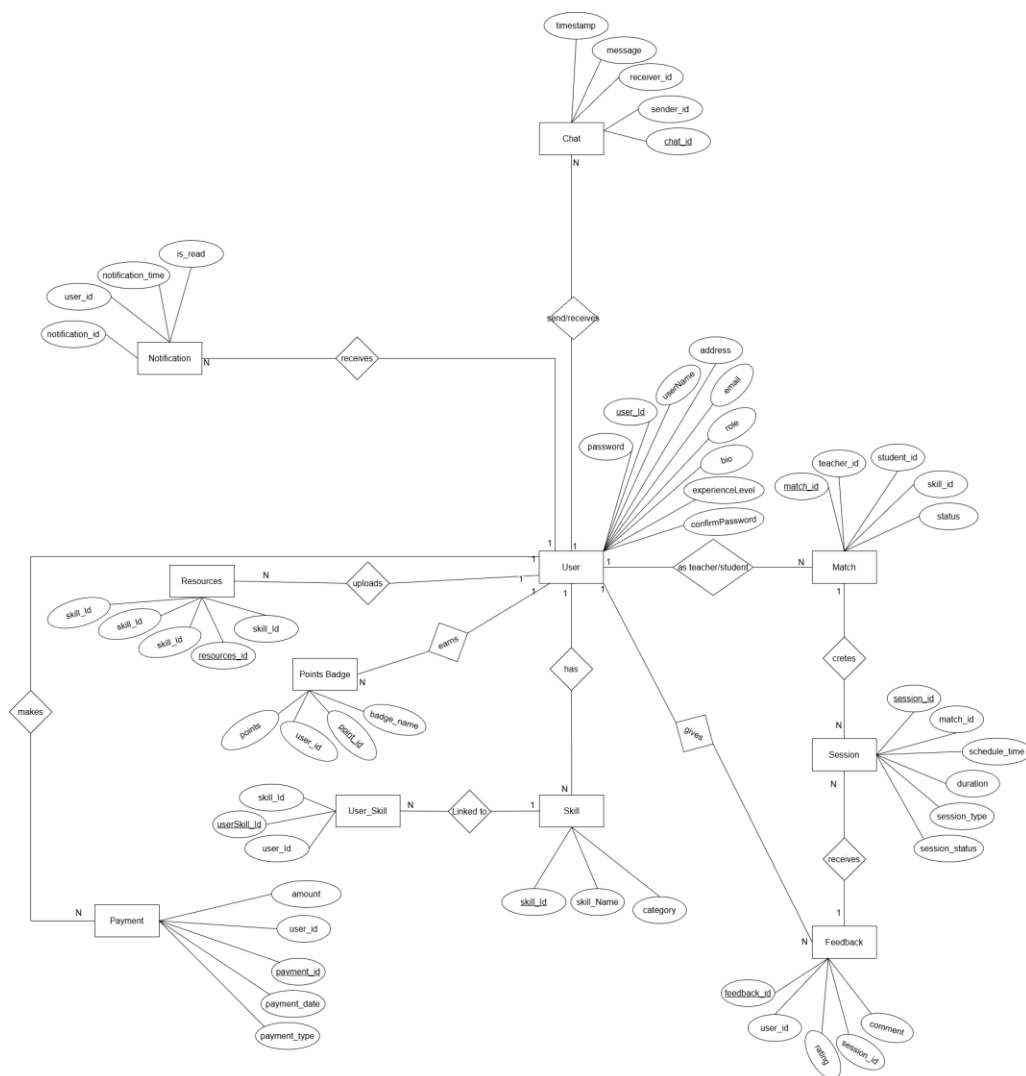


Figure 29: ERD

The ERD gives the database format of Skill Barter System with emphasis on entities, attributes, and their association. User entity is the central entity that links to the Skills via

the User Skill table to handle many to many relationships. The Match entity matches students and teachers and may create various Sessions to have scheduled learning. Learning materials that are stored in the entity of Resources are uploaded by teachers and communication among users is facilitated in the entity of Chat. The Premium upgrades and transactions confirmed by the admin are tracked in the Payment entity, the Notifications enable users to see what happens in the system and the Feedback archive the ratings and comments received after the sessions, which creates quality and trust within the platform.

Sequences Diagram:

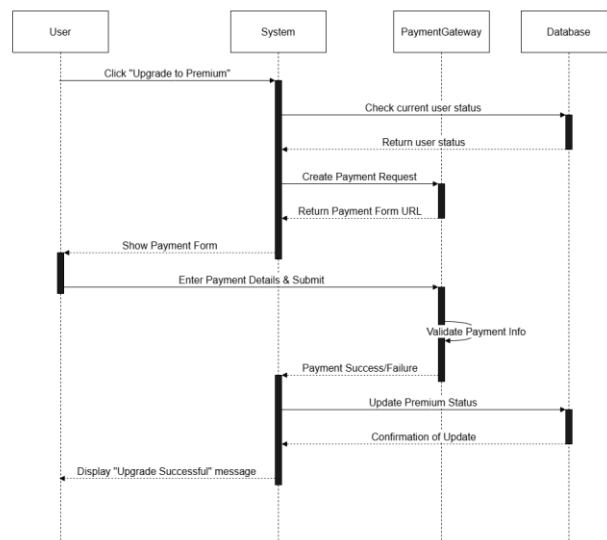


Figure 30: Sequences Diagram

The sequence diagram illustrates the stepwise interaction between the user, system, payment gateway and the database when a user decides to upgrade to a premium account. It would start with the user clicking on the option of "Upgrade to Premium," and the system would verify the status of the user, based on the database, to check if the user is eligible. After the confirmation, the system generates a payment request and forwards it to the payment gateway, which responds to the user by sending them a payment form. The user inserts payment information and payment gateway will verify the payment and returns success or failure information. In case of successful payment, the system will

change the user premium in the database and show the user finally a confirmation message, and the process of upgrading the system is complete.

Flowchart:

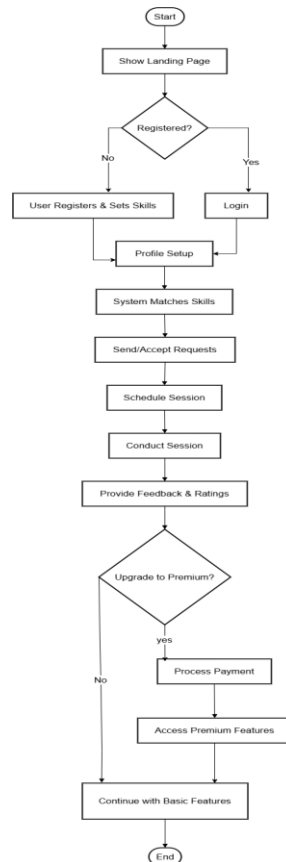


Figure 31: Flowchart

The flowchart shows the general user experience in the skill barter platform, which begins with the landing page and goes through various phases of interaction. It initially verifies the membership of the user; otherwise, the user registers and specifies their skills, whereas authenticated users simply log in. Once the profiles are set up, the system compares the users according to skills, give user the opportunity to send or receive requests, plan sessions and have skill sharing sessions. After a session is finished it is rated and rated by the user. Upon completion of the process, the user is offered alternatives of upgrading to premium whereby paying allows him/her to access premium features or remain on basic features in the platform in case they do not want to upgrade.

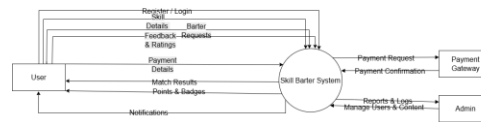
DFD 0:

Figure 32: DFD 0

The Level 0 DFD gives an overview of the Skill Barter System on a high level by depicting the flow of data between the external and the entire system. The user engages with the system through registration details, skills, requests, feedback and payment details and the responses include match results, notifications, ratings and rewards. All the payment related information such as payment requests and confirmations is done in the payment gateway making transactions secure. Also, the system uses the admin to communicate with the system to maintain users, resolve conflict, and manage the running of the system. This diagram is more concerned with the data flow than the process steps or time which presents a clear picture of the data flow of information in and out of the system.

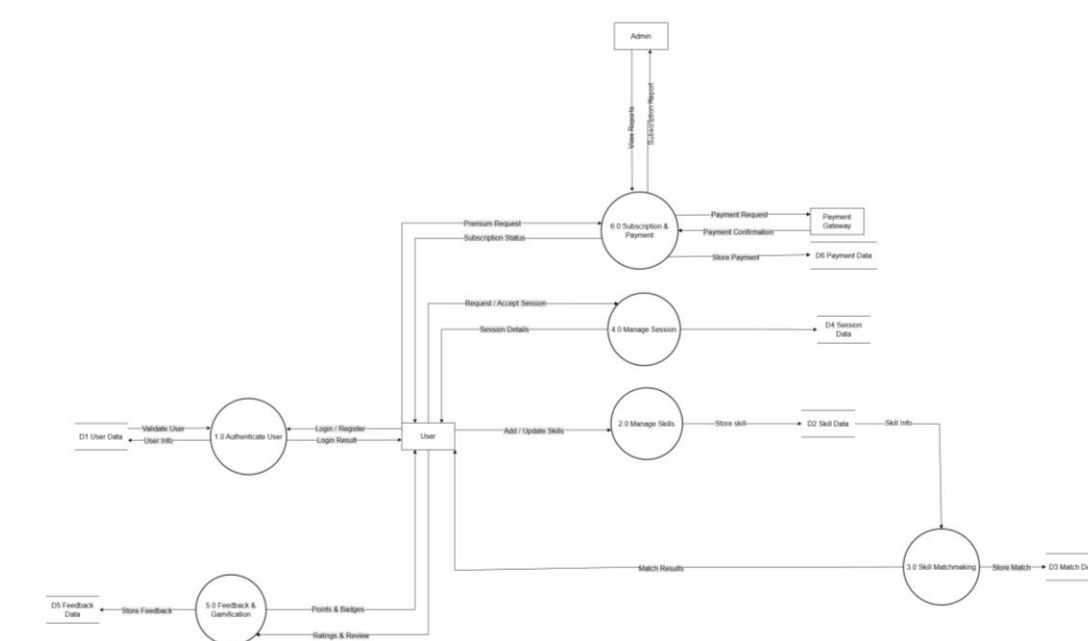
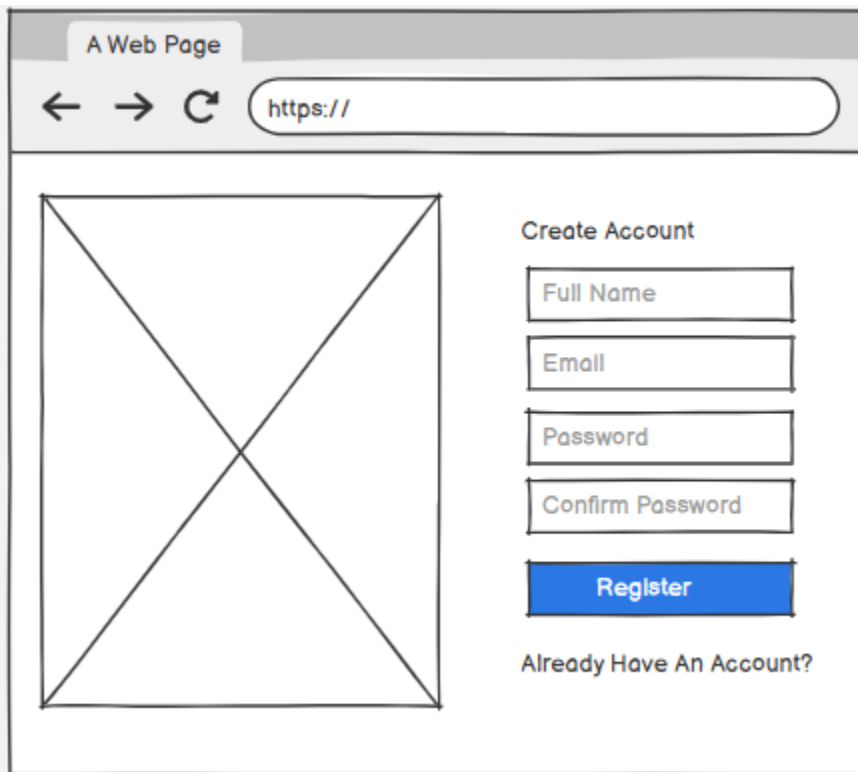
DFD1:

Figure 33: DFD1

6.3 Wireframe



The wireframe depicts a web browser window titled "A Web Page". The address bar shows "https://". The main content area is divided into two sections. On the left is a large square placeholder with a diagonal 'X'. On the right is a registration form titled "Create Account". The form contains four input fields: "Full Name", "Email", "Password", and "Confirm Password". Below these fields is a blue "Register" button. At the bottom of the form area is the text "Already Have An Account?".

Figure 34: Register page's wireframe

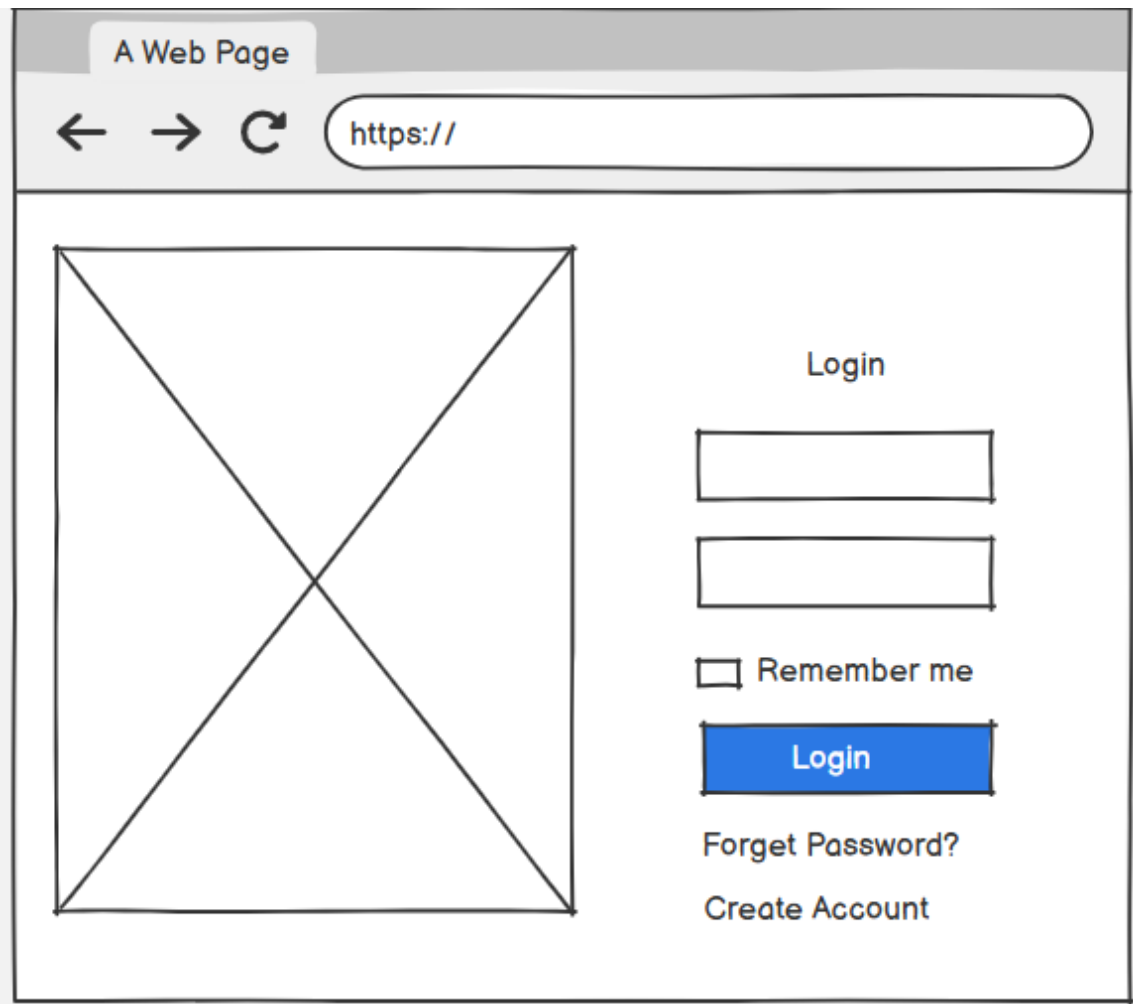


Figure 35: Login page's wireframe

6.4 Implementation Work Completed

After the design phase, the first implementation was done on basic authentication functionality. The User Registration module was created to enable new users to open an account safely. This was done by input validation to combat inaccurate data and avoid invalid submissions. User login module was also finished through which registered users could access the system safely. The logic of backend authentication was introduced that authenticates user credentials and generate sessions. At the front end, the design was done in a way that created attractive responsive UI designs on both the login and registration page, providing smooth user experience on any device.

6.4.1 Code

```

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Login | Skill Barter System</title>
    <link rel="stylesheet" href="{{ asset('css/auth.css') }}">
</head>
<body>

<div class="page-center">

    <div class="auth-card">
        <div class="card-left">
            
        </div>

        <div class="card-right">
            <form method="POST" action="{{ route('login') }}">
                @csrf
                <h2>Login</h2>
                <input type="email" name="email" placeholder="Email"
                    value="{{ old('email') }}" required>
                @error('email') <small class="error">{{ $message }}</small> @enderror
                <input type="password" name="password" placeholder="Password" required>
                @error('password') <small class="error">{{ $message }}</small> @enderror
                <div class="remember">
                    <input type="checkbox" name="remember" id="remember">
                    <label for="remember">Remember Me</label>
                </div>
                <button type="submit">Login</button>
                <div class="links">
                    <a href="{{ route('password.request') }}">Forgot Password?</a><br>
                    <a href="{{ route('register') }}">Create Account</a>
                </div>
            </form>
        </div>
    </div>

</div>

</body>

```

Figure 36: login code from login.blade.php

```

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Register | Skill Barter System</title>
    <link rel="stylesheet" href="{{ asset('css/auth.css') }}">
</head>
<body>

<div class="page-center">
    <div class="auth-card">
        <div class="card-left">
            
        </div>
        <div class="card-right">
            <form method="POST" action="{{ route('register') }}">
                @csrf
                <h2>Create Account</h2>
                <input type="text" name="name" placeholder="Full Name"
                    value="{{ old('name') }}" required>
                @error('name') <small class="error">{{ $message }}</small> @enderror

                <input type="email" name="email" placeholder="Email"
                    value="{{ old('email') }}" required>
                @error('email') <small class="error">{{ $message }}</small> @enderror

                <input type="password" name="password" placeholder="Password" required>
                @error('password') <small class="error">{{ $message }}</small> @enderror

                <input type="password" name="password_confirmation"
                    placeholder="Confirm Password" required>

                <button type="submit">Register</button>
                <div class="links">
                    <a href="{{ route('login') }}">Already have an account?</a>
                </div>
            </form>
        </div>
    </div>
</div>

```

Figure 37: Register code from register.blade.php

```

<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration {
    public function up(): void
    {
        Schema::create('users', function (Blueprint $table) {
            $table->id();
            $table->string('name',150);
            $table->string('email',200)->unique();
            $table->string('password');
            $table->enum('role', ['user','admin','moderator']->default('user'));
            $table->text('bio')->nullable();
            $table->string('avatar')->nullable();
            $table->rememberToken();
            $table->timestamps();
        });
    }

    public function down(): void
    {
        Schema::dropIfExists('users');
    }
};

```

Figure 38: Migration file to create user

```

Route::middleware('guest')->group(function () {
    Route::get('/register', [RegisteredUserController::class, 'create']->name('register'));
    Route::post('/register', [RegisteredUserController::class, 'store']);

    Route::get('/login', [AuthenticatedSessionController::class, 'create']->name('login'));
    Route::post('/login', [AuthenticatedSessionController::class, 'store']);
});

```

Figure 39: Route for login and register

6.4.2 Testing

Register:

- Tested with valid inputs to confirm successful registration.

Table 4: Test1 Registration with valid inputs

Item	Description
Objective	To verify that a new user can register successfully.
Action	Enter valid user details and submit registration.
Expected Outcome	The system should create a new user account.
Actual Result	The user account is created and stored in the database.
Conclusion	Success

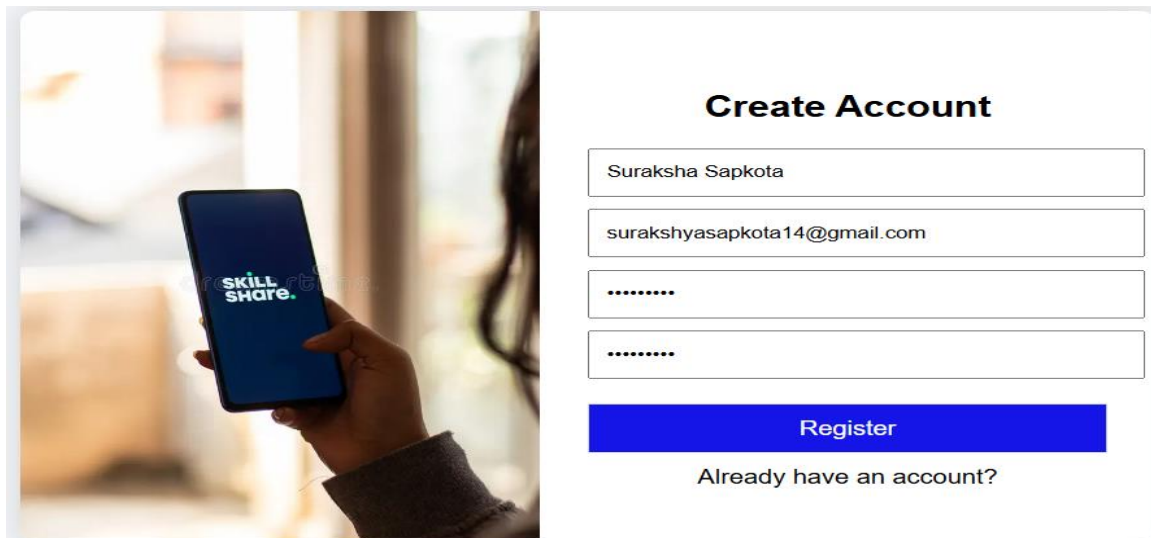


Figure 40: Insert right value to successful registration

- Tested with invalid empty fields, and incorrect password.

Table 5: Test2 Registration with empty fields

Item	Description
Objective	To verify system validation for incomplete registration details.
Action	Submit the registration form with empty fields.
Expected Outcome	The system should display an error message.
Actual Result	An error message has been shown.
Conclusion	Success

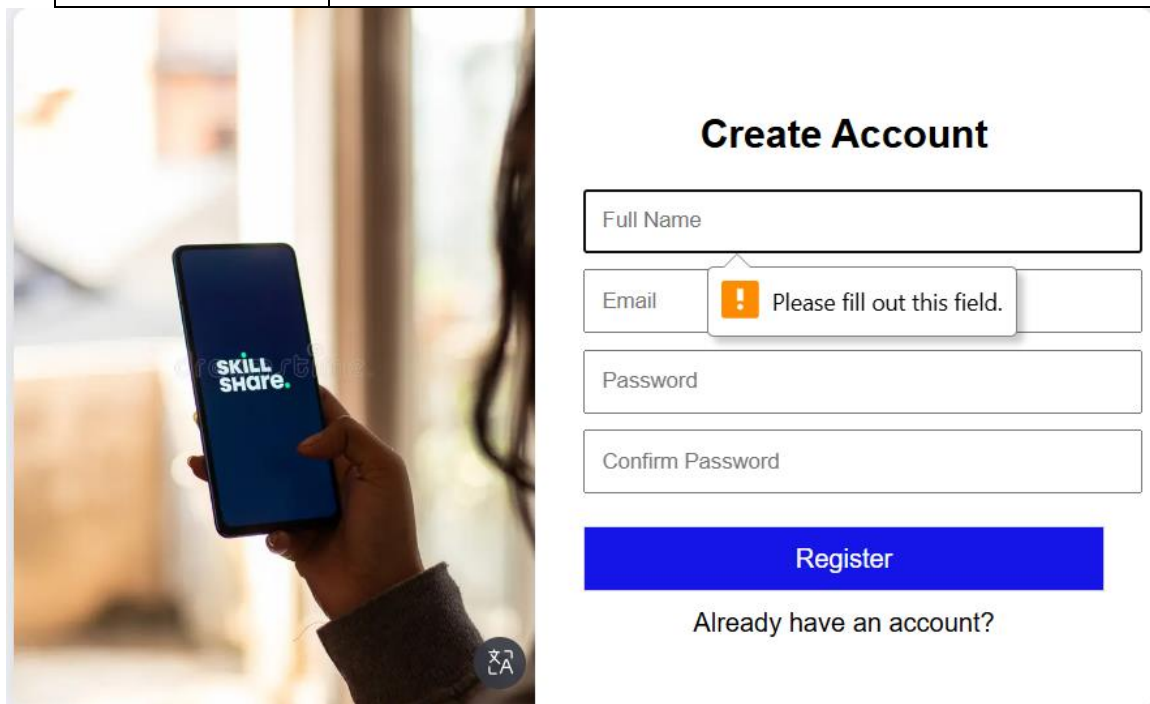


Figure 41: Testing with empty fields

- Verified data storage in the database.

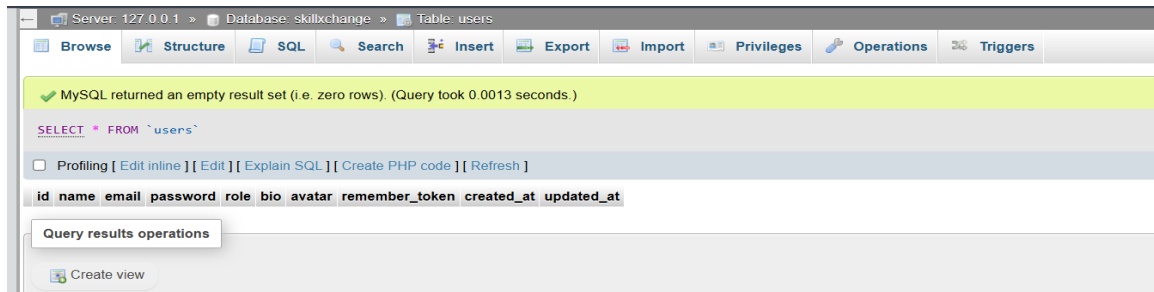


Figure 42: Before Registration

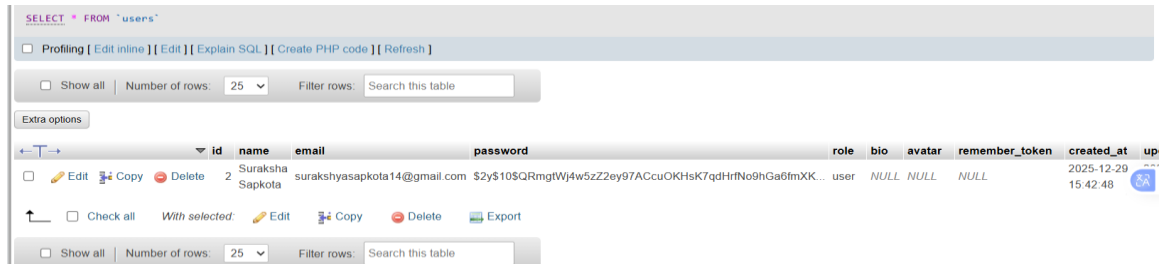


Figure 43: After Registration

Login:

- Login page is open after successful registration

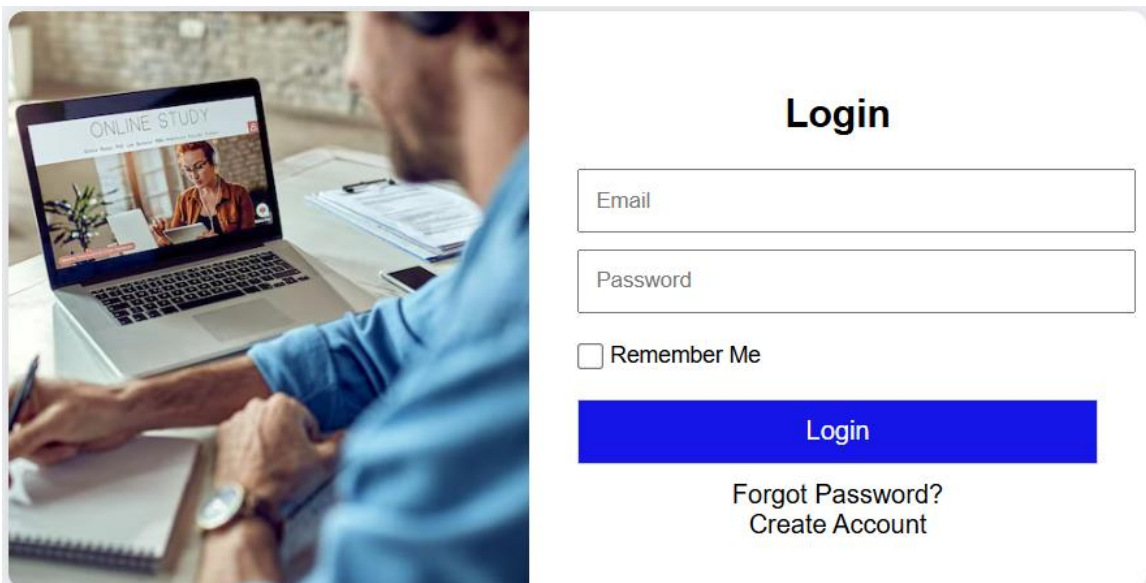


Figure 44: Login Page

- Tested login with incorrect credentials.

Table 6: Test3 Login with wrong credentials

Item	Description
Objective	To verify that the system rejects invalid credentials
Action	Enter an incorrect email or password into the login form.
Expected Outcome	The system should display an error message.
Actual Result	An error message has been shown.
Conclusion	Success

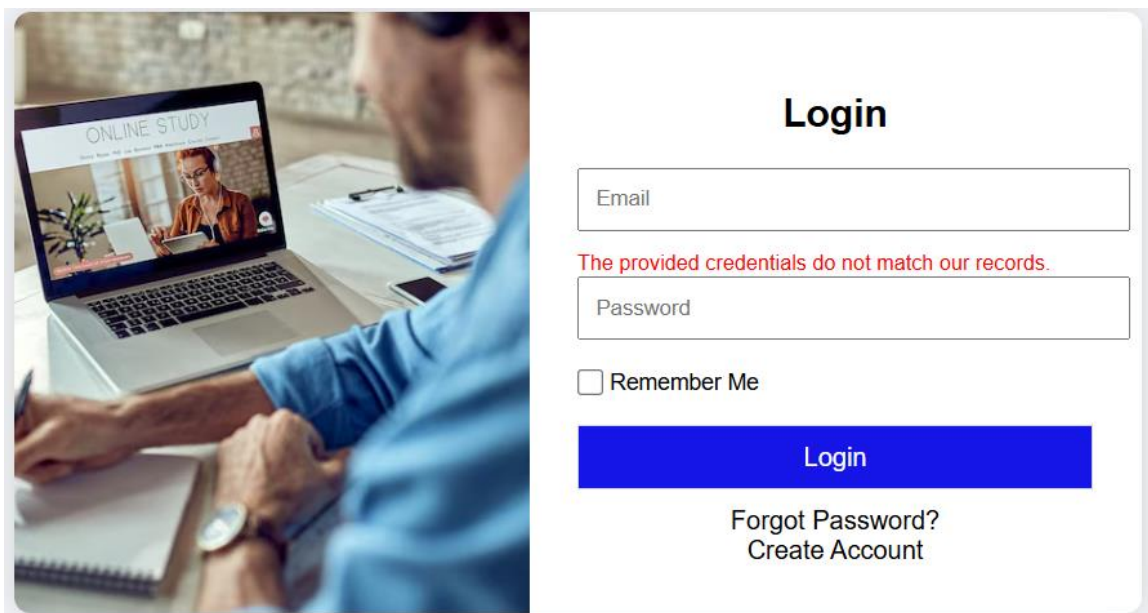


Figure 45: Testing with incorrect values

- Tested login with correct credentials.

Table 7: Test4 Login with correct credentials

Item	Description
Objective	To verify that a new user can login successfully.
Action	Enter valid user details into the login form.
Expected Outcome	The system should authenticate the user and redirect to the homepage.

Actual Result	The user is logged in.
Conclusion	Success

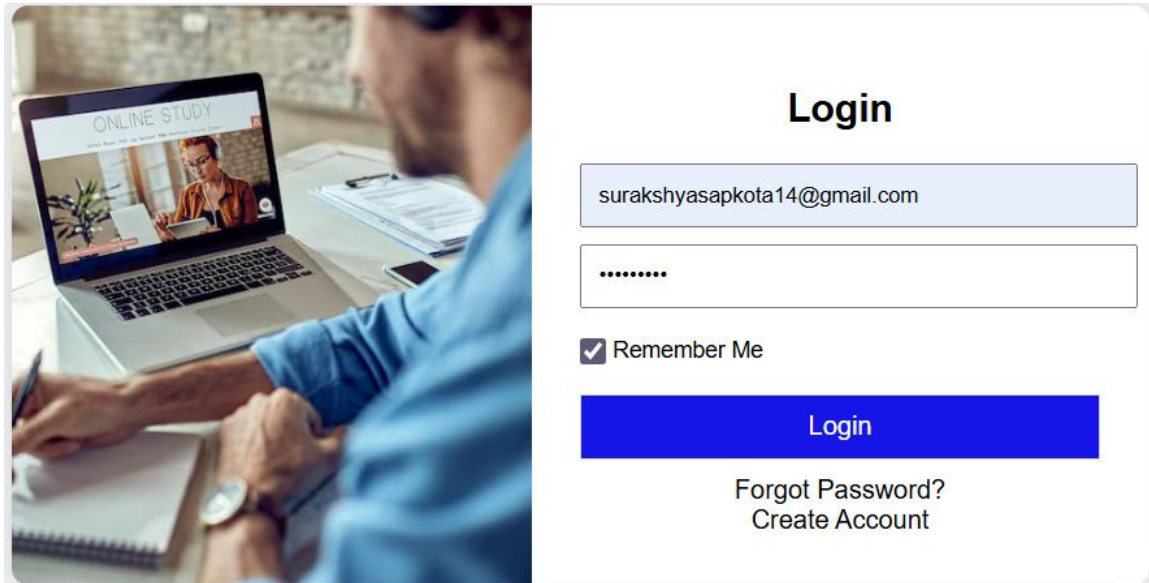


Figure 46: Insert correct value for testing

- After successfully Login home page is opened with user profile

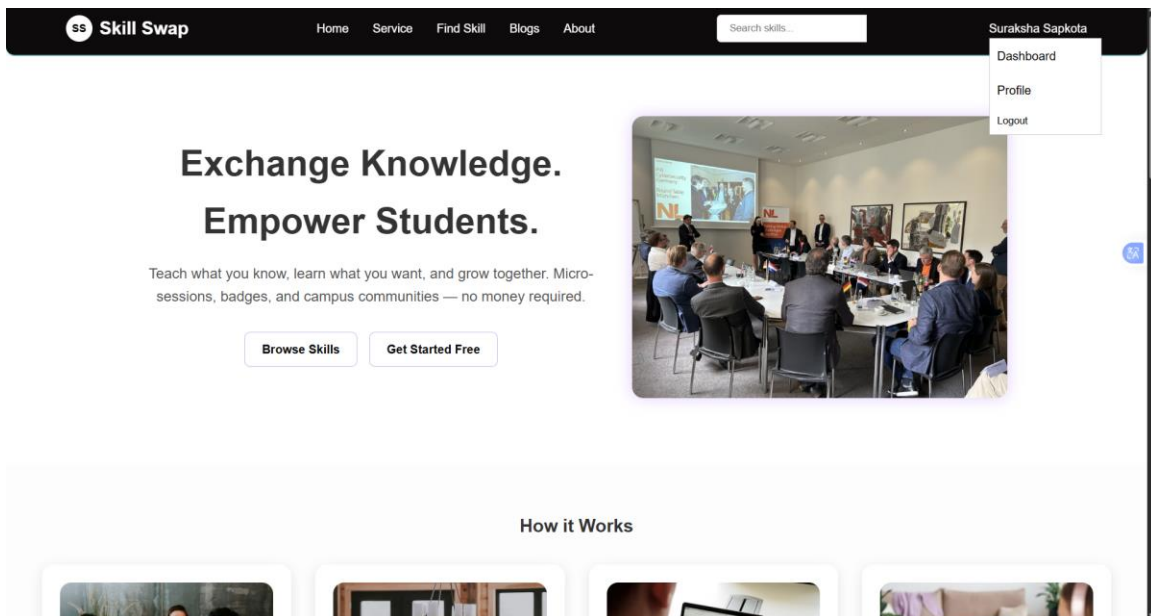


Figure 47: Home page

 Dashboard

Suraksha Sapkota ▾

Profile

Profile Information

Update your account's profile information and email address.

Name

Email

SAVE

Update Password

Ensure your account is using a long, random password to stay secure.

Current Password

New Password

Confirm Password

SAVE

Delete Account

Once your account is deleted, all of its resources and data will be permanently deleted. Before deleting your account, please download any data or information that you wish to retain.

DELETE ACCOUNT

Figure 48: user profile

6.5 System Requirement Specifications (SRS)

6.5.1 Introduction

- Purpose:

This document is a document that outlines the system requirements of SkillXChange, which is a web-based skills barter system where individuals can acquire or give skills without the use of money. The primary idea behind the system is to encourage peer-to-peer-learning, collaboration and knowledge sharing through the provision of safety and orderly environment of skills. This specification is aimed at the developers, testers, supervisors, and the academic reviewers to learn the functionality and limitations of the system.

- Scope
 - User registration and login system.
 - User profile and skill management (skills to teach and learn).
 - Browsing and matching skills between users.
 - Request and approval of skill exchange.
 - User-to-user communication during skill exchange.
 - Post-skilling feedback and rating.
- Definitions, Acronyms, and Abbreviations
 - SRS – Software Requirements Specification
 - UML – Unified Modeling Language
 - DFD – Data Flow Diagram
 - ERD – Entity Relationship Diagram

6.5.2 Overall Description

- Product Perspective:

SkillXChange is an independent web-based application, and that is meant to facilitate skill exchange without any money transactions. The system is used by the users with the Web interface, and all the information concerning the users, skills, exchanges, and feedback is stored and managed in the central database. The system is client-server based on the MVC pattern of design.

- User Classes and Characteristics
 - Admin
 - Manage user
 - Manage skills
 - Monitor activity
 - Handle policy
 - User
 - Register and Log In
 - Manage profile
 - Adds skills to teach and learn
 - Skill exchange
 - Provide feedback
- Operating Environment
 - Web browsers
 - Internet connection
 - Server with PHP (Laravel Framework)
 - MySQL database
- Design and Implementation Constraints
 - System must be web-based
 - Internet connection required
 - Data must store
 - Only registered users can access system features

6.5.3 Functional Requirements

- User Authentication
 - The system should enable its users to register using valid information.
 - The system will enable users to log in with email/username and password.
 - The system must ensure that the credentials provided by the user are validated before access is allowed.

- Profile and Skill Management
 - The system should enable users to create and modify their profiles.
 - The system should enable users to enter, modify, and remove skills.
 - The system should be capable of storing skill information like category and description.
- Skill Browsing and Matching
 - The system should enable browsing skills by users.
 - The system should support searching of skills based on categories or keywords.
 - Users will see relevant skill correlations within the system.
- Skill Exchange Requests
 - The system will enable the sending of skill exchange requests.
 - It shall be possible in this system for a user to accept or reject requests.
 - The system should record the status of every exchange.
- Communication
 - The system should enable users to communicate within the system.
 - Messages will be associated with exchange sessions.
- Feedback and Rating
 - The system shall enable feedback from users following an exchange.
 - The system shall be capable of storing ratings based on user reliability.
 - Admin should be able to view feedback.

6.5.4 Non-functional Requirements

- Performance
 - The system will take 2-3 seconds to load pages.
 - The system must support several users at once.
- Security
 - Password must be encrypted for users
 - Role-Based Access Control must be implemented by the system.
 - Secure protocols (HTTPS) must be used for information transport.

- Usability
 - The system should be equipped with an easy-to-use interface
 - Both desktop computers and mobile phones should be able to access the system.
 - The error messages will be simple to read and comprehend.
- Reliability
 - The data integrity will be supported by the system.
 - Minor errors will be properly corrected by the system.

6.5.5 Future Enhancements

- AI-based skill matching
- Gamification and rewards system
- Premium membership
- Advanced analytics & reporting

7. FURTHER WORK

This section will describe the remaining tasks and upcoming upgrades to the Skill Barter System. Due to time constraints, academic focus considerations, and the need to establish a foundational system before exploring additional enhancements, certain features included in the first project proposal submission had to be modified. As a result, the development process will be progressive, starting with system design, user interface components, and authentication.

7.1 Reflection on Changes from the Original Proposal

A full skill trading system with real-time chat, credit transaction systems, rating facilities, and full skill trading was to be built in a single development cycle, according to the original design. However, it has been reduced to a single topic because of academic time constraints and the complexity in developing multiple interacting systems.

This section will tell about the remaining task and future improvements in the Skill Barter System:

- Limited Development Time in the academic semester.
- The need to focus on basic functionalities and avoid sophistication.
- Ensure stability in a system before adding financial/credit-based modules.
- System logic and user flow validation by increment.

The project is currently concentrating on user authentication, system modeling, user interface design, and basic backend implementation. The more complex tasks of skill exchange and messaging have been planned for implementation during the later stages of the project.

7.2 Remedial Plans Adopted

To overcome these limitations, an incrementally developed system design methodology has been employed in this project. The major functionalities like user registration, user login, identification of roles, and validation processes are included in the project design initially. The system design concepts like detailed UML diagram, DFD diagram, ER diagram, flow chart, sequence diagram, and collaboration diagrams are employed to provide proper understanding to the system design.

A user interface design was completed using wireframes and prototypes based on usability principles that came out of research. Backend validation was incorporated with a

focus on avoiding submitting data incorrectly. It is important since it ensures that whenever a new module is added in the future, integration is smooth.

7.3 Further Work to be Done Using Incremental Development

Development in the future will be carried out incrementally so that each module can be designed independently.

- Increment 1: Authentication and access control

This phase has been completed as planned. It includes session handling for user authentication, login, and registration.

- Increment 1: Management of Skills

Skills can be added, changed, and removed by users. The talents are tied to user profiles, searchable, and arranged. To guarantee accurate description and classification, the skills are validated.

- Increment 3: Skill Matching and Barter Logic

This update will focus on matching users according to the talents that can be provided and those that are needed. Barter requests, acceptances, rejections, and tracking of ongoing exchanges will all be managed by the system.

- Increment 4: Credit / Point-Based Exchange System

There would be a credit system to enable fair value trading. Credits would be earned, spent, and documented on a secure log under this system.

- Increment 5: Communication and Collaboration Features

To allow users to communicate while exchanging talents, this phase will build message and student communication functionality. The student will be informed of the various actions carried out by user notifications.

- Increment 6: Rating, Feedback, and Trust Management

After completing a skills exchange, users will be able to review and rate. This will have an impact on the platform's display and trust levels.

➤ Increment 7: Testing, Security, and Performance Optimization

There will be lots of testing, including user experience, authentication, and validation testing. Access control, safe data management, and encrypted credentials are some of the security features that will be enhanced.

7.4 Deployment and Documentation

The Skill Barter System will run on a web server environment that supports Laravel and an Relational Database. The documentation will cover user manuals, administrator manuals, explanations of the system architecture, and testing reports. There will be screenshots and guides that facilitate ease of use.

7.5 Alignment with Proposed Gantt Chart

To show progressive progression over a linear temporal approach, the Gantt diagram has been revised. This is due to the sequential design of each increment in incremental development, which permits overlap between design, implementation, and testing.

7.6 Expected Outcomes

The Skill Barter System will provide a safe and welcoming environment for skill trade after the remaining tasks are finished. Users will be able to register, manage talents, exchange services fairly through credits, communicate clearly, and gain the trust of others through reviews and ratings. Scalability and improvement will be made possible by the gradual approach.

8. CONCLUSION

This interim report explains the ongoing progress and plans for the SkillXChange project, an online skill barter platform which allows users to share knowledge and skills without the involvement of money. The system focuses on building a cooperative digital space where people can interact with others who possess skills they need and promote learning from and teaching each other. During this period, milestones were reached, including requirement gathering, background and journal research, system modeling with UML, DFD, ERD, flowchart, and other design diagrams, as well as the development of primary authentication features such as user registration and login. These primary components create a protective and organized foundation for the system to develop. The project plans to include additional features such as skill-matching artificial intelligence, real-time communication, and scheduling along with as feedback and rating systems and gamification in the form of points, badges, and leaderboards to increase user engagement. Moreover, the addition of a Premium Membership model is planned to offer advanced features and added value for the more active users. Overall, SkillXChange aims to be a straightforward, safe, and efficient platform for the exchange of knowledge, foster collaboration and learning, and support continuous learning. The incrementally planned development assures the system's extensibility and dependability, as well as making it ready for deployment in a live environment.

9. REFERENCES

- Bennett, L., 2024. GURU99. [Online] Available at: <https://www.guru99.com/software-engineering-prototyping-model.html> [Accessed 28 Oct 2025].
- DSouza, D., 2024. Hevo. [Online] Available at: <https://hevodata.com/learn/xampp-mysql/#:~:text=XAMPP%20is%20an%20open%2Dsource,PHP%20and%20Perl%20programming%20languages.> [Accessed 22 08 2024].
- Sachan, D., 2024. Experience SCALER. [Online] Available at: <https://www.scaler.com/topics/incremental-model-in-software-engineering/> [Accessed 28 Oct 2025].
- Purdila, A., 2023. envatotuts+. [Online] Available at: <https://webdesign.tutsplus.com/introduction-to-figma-free-beginners-course> [Accessed 28 Oct 2025].
- tutorialspoint, 2024. tutorialspoint. [Online] Available at: https://www.tutorialspoint.com/laravel/laravel_overview.htm [Accessed 28 Oct 2025].
- UCHENDU, C. C., 2021. webopedia. [Online] Available at: <https://www.webopedia.com/definitions/visual-studio-code/> [Accessed 28 Oct 2025].